Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho

PCC104 - Projeto e Análise de Algoritmos

Henrique Ulhôa de Faria
Professor: Pedro Silva

Ouro Preto
9 de novembro de 2026

Sumário

1 Introdução **2**
- 1.1 Especificações do problema . . . 2
- 1.2 Considerações iniciais . . . 2
- 1.3 Ferramentas utilizadas . . . 2
- 1.4 Especificações da máquina . . . 2
- 1.5 Instruções de compilação e execução . . . 3

2 Metodologia **4**
- 2.1 Análise dos enunciados . . . 4
- 2.2 Projeto e implementação dos algoritmos . . . 4
- 2.3 Análise de complexidade . . . 4
- 2.4 Testes e validação . . . 4
- 2.5 Organização e documentação . . . 4

3 Tarefa 1 – Multiplicação de Matrizes **5**
- 3.1 Descrição do problema . . . 5
- 3.2 Estratégia de solução . . . 5
  - 3.2.1 Tratamento de matrizes de ordem arbitrária . . . 5
- 3.3 Implementação . . . 6
  - 3.3.1 Estrutura de dados . . . 6
  - 3.3.2 Operações básicas . . . 6
  - 3.3.3 Funções auxiliares . . . 6
  - 3.3.4 Multiplicação recursiva . . . 7
  - 3.3.5 Função principal de multiplicação . . . 7
  - 3.3.6 Programa principal . . . 7
- 3.4 Análise de complexidade . . . 7
  - 3.4.1 Complexidade de tempo . . . 7
  - 3.4.2 Complexidade de espaço . . . 8
  - 3.4.3 Análise do preenchimento com zeros . . . 9
- 3.5 Experimentos e resultados . . . 9
  - 3.5.1 Metodologia dos testes . . . 9
  - 3.5.2 Casos de teste . . . 9
  - 3.5.3 Teste 01 – Matrizes de ordem 2 . . . 10
  - 3.5.4 Teste 02 – Caso base . . . 10
  - 3.5.5 Teste 03 – Ordem não potência de dois . . . 10
  - 3.5.6 Teste 04 – Matriz identidade . . . 10
  - 3.5.7 Análise dos resultados . . . 11
- 3.6 Considerações da tarefa . . . 11

4 Tarefa 2 – Algoritmo de Dijkstra utilizando vetores **12**
- 4.1 Descrição do problema . . . 12
- 4.2 Estratégia de solução . . . 12
  - 4.2.1 Representação do grafo . . . 13
  - 4.2.2 Seleção dos vértices . . . 13
  - 4.2.3 Reconstrução dos caminhos . . . 13
- 4.3 Implementação . . . 13
  - 4.3.1 Estruturas de dados . . . 13
  - 4.3.2 Operações básicas do grafo . . . 15
  - 4.3.3 Implementação do algoritmo de Dijkstra . . . 16
  - 4.3.4 Reconstrução dos caminhos . . . 17
  - 4.3.5 Apresentação de todos os caminhos . . . 18
  - 4.3.6 Programa principal . . . 18
- 4.4 Análise de complexidade . . . 20
  - 4.4.1 Complexidade de tempo . . . 20
  - 4.4.2 Complexidade de espaço . . . 21

4.4.3 Complexidade da reconstrução dos caminhos . . . 21
4.4.4 Complexidade das operações do grafo . . . 22
4.5 Experimentos e resultados . . . 22
4.5.1 Metodologia experimental . . . 22
4.5.2 Resultados dos experimentos . . . 23
4.5.3 Comparação entre os grafos de Roma e Nova York . . . 24
4.5.4 Análise dos resultados . . . 26
4.6 Considerações da tarefa . . . 27

5 Tarefa 3 – Algoritmo de Dijkstra utilizando Listas Encadeadas 29
5.1 Descrição do problema . . . 29
5.2 Estratégia de solução . . . 29
5.2.1 Representação do grafo . . . 29
5.2.2 Execução do algoritmo . . . 29
5.3 Implementação . . . 30
5.3.1 Estruturas de dados . . . 30
5.3.2 Inserção de arestas . . . 30
5.3.3 Adaptação do algoritmo de Dijkstra . . . 31
5.3.4 Programa principal . . . 31
5.4 Análise de complexidade . . . 31
5.4.1 Complexidade de tempo . . . 31
5.4.2 Complexidade de espaço . . . 32
5.4.3 Comparação entre representações . . . 32
5.5 Experimentos e resultados . . . 32
5.5.1 Metodologia dos testes . . . 32
5.5.2 Resultados experimentais . . . 33
5.5.3 Análise comparativa dos resultados . . . 33
5.6 Considerações da tarefa . . . 34

6 Tarefa 4 – Algoritmo de Dijkstra utilizando Fila de Prioridade 35
6.1 Descrição do problema . . . 35
6.2 Estratégia de solução . . . 35
6.3 Implementação . . . 35
6.4 Análise de complexidade . . . 36
6.4.1 Complexidade das operações do heap . . . 36
6.4.2 Complexidade de tempo . . . 36
6.4.3 Complexidade de espaço . . . 36
6.4.4 Comparação com as implementações anteriores . . . 37
6.5 Experimentos e resultados . . . 37
6.5.1 Testes de funcionamento . . . 37
6.5.2 Metodologia dos experimentos . . . 37
6.5.3 Tempos de execução . . . 38
6.5.4 Operações da fila de prioridade . . . 38
6.5.5 Comparação de desempenho . . . 38
6.5.6 Análise dos resultados . . . 39
6.6 Considerações da tarefa . . . 39

7 Tarefa 5 – Título a definir 40

8 Tarefa 6 – Título a definir 41

9 Tarefa 7 – Título a definir 42

10 Tarefa 8 – Título a definir 43

11 Tarefa 9 – Título a definir 44

12 Considerações Finais 45

Lista de Tabelas

1 Casos de teste da multiplicação de matrizes. 9
2 Características dos grafos utilizados nos experimentos. 25
3 Tempos de execução do algoritmo de Dijkstra para o grafo de Roma. 25
4 Tempos de execução do algoritmo de Dijkstra para o grafo de Nova York. 25
5 Resumo dos tempos de execução dos grafos avaliados. 25
6 Comparação das razões de crescimento entre os grafos. 25
7 Instâncias utilizadas nos experimentos. 33
8 Tempos de execução para o grafo de Roma (s). 33
9 Tempos de execução para o grafo de Nova York (s). 33
10 Comparação das implementações de Dijkstra. 37
11 Tempos de execução da Tarefa 4. 38
12 Operações registradas na Tarefa 4. 38
13 Comparação dos tempos médios de Dijkstra. 38
14 Fatores de aceleração da Tarefa 4. 39

Lista de Códigos Fonte

1 Definição do TAD Matriz. 6
2 Caso base da multiplicação recursiva. 7
3 Estrutura utilizada para representar um vértice. 13
4 Estrutura utilizada para representar uma aresta. 14
5 Estrutura utilizada para representar o grafo. 14
6 Estrutura utilizada para armazenar os resultados de Dijkstra. 14
7 Inserção de uma aresta utilizando índices. 15
8 Exemplo de entrada no formato DIMACS. 15
9 Inicialização dos vetores do algoritmo de Dijkstra. 16
10 Seleção do vértice de menor distância. 16
11 Percurso e relaxamento das arestas. 17
12 Reconstrução do caminho por predecessores. 18
13 Formato de saída dos caminhos mínimos. 18
14 Exemplo de saída de um caminho mínimo. 18
15 Medição do tempo de execução do algoritmo. 19
16 Execução do programa com o grafo de Roma. 19
17 Estrutura de uma aresta na lista encadeada. 30
18 Estrutura de um vértice com lista de adjacência. 30
19 Estrutura do grafo com listas encadeadas. 30
20 Inserção de uma aresta no início da lista. 30
21 Percurso e relaxamento das listas encadeadas. 31
22 Estruturas utilizadas na fila de prioridade. 35

Sumário

1 Introdução

Este relatório apresenta o desenvolvimento das atividades propostas no Trabalho Prático da disciplina PCC104 – Projeto e Análise de Algoritmos. O trabalho contempla a implementação, a análise de complexidade e a validação de algoritmos, com o objetivo de relacionar os conceitos teóricos estudados na disciplina às suas aplicações práticas.

As soluções são desenvolvidas na linguagem de programação C, considerando aspectos como organização do código, correção dos resultados, eficiência computacional e utilização de estruturas de dados adequadas.

1.1 Especificações do problema

O Trabalho Prático é composto por diferentes tarefas de implementação de algoritmos. Cada tarefa apresenta um problema específico, com requisitos de entrada, saída e processamento que devem ser respeitados.

Para cada problema, busca-se construir uma solução correta, analisar sua complexidade de tempo e espaço e verificar seu funcionamento por meio de testes. As especificações particulares de cada atividade são apresentadas nos respectivos capítulos deste relatório.

1.2 Considerações iniciais

O relatório está organizado em uma parte introdutória, uma seção de metodologia geral e capítulos dedicados individualmente às tarefas. Cada capítulo de tarefa apresenta a descrição do problema, a estratégia de solução, os aspectos relevantes da implementação, a análise de complexidade, os testes e os resultados obtidos.

O código-fonte é desenvolvido em C e organizado em arquivos separados, conforme as necessidades de cada atividade. A documentação é elaborada em LaTeX, utilizando a plataforma Overleaf.

1.3 Ferramentas utilizadas

As ferramentas empregadas no desenvolvimento e na validação das implementações incluem:

- **Linguagem C**: implementação dos algoritmos e das estruturas de dados.
- **Visual Studio Code**: edição e organização dos arquivos de código-fonte.
- **GCC**: compilação dos programas em C.
- **GNU Make**: automatização da compilação e da execução dos testes.
- **MSYS2 UCRT64**: ambiente utilizado para compilação e testes no Windows.
- **Git e GitHub**: controle de versões e armazenamento do projeto.
- **Overleaf e LaTeX**: elaboração e formatação do relatório.

1.4 Especificações da máquina

O desenvolvimento e os testes foram realizados em um computador com as seguintes características, a serem confirmadas:

- **Processador**: Intel I5 11° Geração.
- **Memória RAM**: 24 Ram.
- **Sistema operacional**: Windows 10.
- **Sistema operacional**: Ubuntu.
- **Compilador**: GCC no ambiente MSYS2 UCRT64 [informar versão utilizada].

1.5 Instruções de compilação e execução

Os programas são organizados em diretórios específicos para cada tarefa. Quando houver um arquivo `Makefile`, a compilação poderá ser realizada, a partir do diretório correspondente, com o comando:

Compilação com Make

```
make
```

Na Tarefa 1, também é possível compilar diretamente com o GCC:

Compilação da Tarefa 1

```
gcc -Wall -Wextra -Wpedantic -std=c11 -Iinclude src/main.c src/matriz.c -o tarefa01.exe
```

As principais opções utilizadas são:

- `-Wall`: habilita avisos comuns do compilador.
- `-Wextra`: habilita avisos adicionais.
- `-Wpedantic`: sinaliza construções que não seguem estritamente o padrão selecionado.
- `-std=c11`: seleciona o padrão C11.
- `-Iinclude`: informa o diretório dos arquivos de cabeçalho.
- `-o`: define o nome do arquivo executável.

Para executar a Tarefa 1 utilizando um arquivo de entrada, emprega-se:

Execução da Tarefa 1

```
./tarefa01.exe < tests/entrada01.txt
```

Para executar os testes automatizados definidos no `Makefile`, utiliza-se:

Testes automatizados

```
make test
```

Os comandos de compilação e execução das demais tarefas serão apresentados ou ajustados conforme suas implementações.

2 Metodologia

O desenvolvimento das atividades segue uma sequência de etapas que busca assegurar a correção dos algoritmos e permitir a análise de seu desempenho. Os procedimentos gerais são descritos a seguir; as particularidades de cada solução são detalhadas nos capítulos correspondentes.

2.1 Análise dos enunciados

Inicialmente, são identificados o objetivo de cada problema, os dados de entrada, os resultados esperados e as restrições estabelecidas. Essa etapa orienta a escolha da estratégia algorítmica e a definição das estruturas de dados necessárias.

2.2 Projeto e implementação dos algoritmos

Após a interpretação do problema, é definida uma estratégia de solução, considerando as técnicas estudadas na disciplina. A implementação é realizada em linguagem C, com separação de responsabilidades entre funções e arquivos quando apropriado.

A documentação de cada tarefa descreve as principais funções, as decisões de implementação e os casos especiais tratados. Trechos relevantes de código podem ser incluídos no relatório utilizando o ambiente `lstlisting` disponibilizado pelo modelo.

2.3 Análise de complexidade

A eficiência das soluções é estudada por meio da análise assintótica de tempo e espaço. São identificadas as operações predominantes e, para algoritmos recursivos, formuladas relações de recorrência. Quando aplicável, essas relações são resolvidas por técnicas como o Teorema Mestre ou árvores de recorrência.

A análise teórica considera as hipóteses necessárias a cada algoritmo, que são explicitadas nas respectivas tarefas.

2.4 Testes e validação

As implementações são verificadas por meio de casos de teste que contemplam entradas simples, casos representativos e situações particulares previstas nos enunciados. Os resultados produzidos são comparados com as saídas esperadas.

Quando disponível, um `Makefile` automatiza a compilação e a execução dos testes. A documentação apresenta os casos utilizados e registra os resultados observados, distinguindo a verificação de correção de eventuais medições experimentais de desempenho.

2.5 Organização e documentação

O projeto utiliza Git para o versionamento dos arquivos de implementação e teste. Este relatório reúne a descrição das soluções, as análises de complexidade e os resultados obtidos, mantendo uma estrutura uniforme para facilitar a comparação e a consulta às diferentes tarefas.

3 Tarefa 1 – Multiplicação de Matrizes

3.1 Descrição do problema

A primeira tarefa consiste na implementação de um algoritmo para realizar a multiplicação de duas matrizes quadradas de ordem n, utilizando a técnica de divisão e conquista. Dadas duas matrizes A e B, ambas de dimensão $n \times n$, o objetivo é calcular a matriz resultante $C = A \cdot B$, também de ordem n. Cada elemento da matriz resultante é definido por:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}, \qquad 1 \leq i, j \leq n. \tag{1}$$

A entrada do programa é composta por um inteiro positivo n, que representa a ordem das matrizes, seguido dos n^2 elementos da matriz A e dos n^2 elementos da matriz B. A saída consiste nos elementos da matriz resultante C, organizados em n linhas e n colunas. A solução deve utilizar uma abordagem recursiva baseada em divisão e conquista, decompondo o problema original em multiplicações de submatrizes de menor dimensão.

3.2 Estratégia de solução

Para resolver o problema, foi utilizada a técnica de divisão e conquista, que consiste em dividir um problema em subproblemas menores, resolvê-los recursivamente e combinar seus resultados para obter a solução original.

Na multiplicação de matrizes, essa estratégia é aplicada dividindo cada matriz quadrada em quatro submatrizes de mesma dimensão. Considerando duas matrizes A e B, de ordem n, suas decomposições podem ser representadas por:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}. \tag{2}$$

Cada submatriz possui dimensão $\frac{n}{2} \times \frac{n}{2}$, considerando inicialmente que n é uma potência de dois.

A matriz resultante $C = A \cdot B$ também pode ser representada por quatro submatrizes:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}. \tag{3}$$

Os blocos da matriz resultante são calculados pelas seguintes expressões:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \tag{4}$$
$$C_{12} = A_{11}B_{12} + A_{12}B_{22}, \tag{5}$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}, \tag{6}$$
$$C_{22} = A_{21}B_{12} + A_{22}B_{22}. \tag{7}$$

Dessa forma, cada etapa da recursão realiza oito multiplicações de submatrizes de ordem $n/2$, além das operações de soma e combinação dos resultados parciais.

O caso base da recursão ocorre quando as matrizes possuem ordem $n = 1$. Nesse caso, a multiplicação é realizada diretamente entre seus únicos elementos, encerrando a chamada recursiva.

Após o cálculo dos quatro blocos resultantes, eles são combinados para formar a matriz produto da etapa atual.

3.2.1 Tratamento de matrizes de ordem arbitrária

Como a divisão recursiva utiliza submatrizes de mesma dimensão, foi implementado um procedimento para tratar matrizes cuja ordem não é uma potência de dois.

Inicialmente, determina-se a menor potência de dois maior ou igual à ordem original n:

$$m = 2^{\lceil \log_2 n \rceil}. \tag{8}$$

Em seguida, são criadas duas matrizes auxiliares de ordem m, preenchidas inicialmente com zeros. Os elementos das matrizes originais são copiados para as posições correspondentes dessas matrizes auxiliares. Esse procedimento, denominado preenchimento com zeros (*zero-padding*), permite que as matrizes sejam divididas sucessivamente em quatro blocos de mesma dimensão. Após a multiplicação recursiva, são extraídos os $n \times n$ elementos da região superior esquerda da matriz resultante, obtendo-se o produto correspondente às matrizes originais. Assim, a solução permite multiplicar matrizes quadradas de qualquer ordem positiva, mantendo a estratégia recursiva de divisão e conquista.

3.3 Implementação

A implementação foi desenvolvida na linguagem C, utilizando um Tipo Abstrato de Dados (TAD) para representar e manipular matrizes quadradas.

O código foi organizado em três arquivos principais: `matriz.h`, responsável pela definição da estrutura e declaração das funções; `matriz.c`, que contém a implementação das operações sobre matrizes; e `main.c`, responsável pela leitura dos dados, execução da multiplicação e apresentação do resultado.

3.3.1 Estrutura de dados

A estrutura `Matriz` armazena a ordem da matriz e um ponteiro para seus elementos, representados por um vetor de ponteiros para linhas alocadas dinamicamente.

```
typedef struct {
    int n;
    int **dados;
} Matriz;
```

Código 1: Definição do TAD Matriz.

O campo `n` representa a quantidade de linhas e colunas, enquanto `dados` permite acessar os elementos por meio da expressão `m->dados[i][j]`.

A alocação dinâmica permite criar matrizes de diferentes dimensões durante a execução, o que é necessário para o funcionamento da abordagem recursiva.

3.3.2 Operações básicas

Foram implementadas funções para criação, liberação de memória, leitura e impressão de matrizes.

A função `createMatriz` recebe a ordem da matriz e realiza a alocação dinâmica de sua estrutura e de seus elementos, inicializados com zero.

A função `deleteMatriz` libera a memória alocada, enquanto `readMatriz` e `printMatriz` realizam, respectivamente, a leitura dos elementos e a apresentação da matriz.

Essas operações são utilizadas tanto pelo programa principal quanto pelas funções auxiliares da multiplicação.

3.3.3 Funções auxiliares

Para implementar a estratégia de divisão e conquista, foram desenvolvidas três funções auxiliares:

- `subMatriz`: cria uma nova matriz contendo uma região quadrada da matriz original, definida pela posição inicial e pela dimensão desejada.
- `somaMatriz`: recebe duas matrizes de mesma ordem e retorna uma nova matriz contendo a soma de seus elementos correspondentes.
- `juntaMatriz`: combina quatro submatrizes em uma única matriz quadrada, posicionando cada bloco no quadrante correspondente.

Essas funções permitem dividir as matrizes de entrada, combinar os produtos parciais e reconstruir o resultado de cada chamada recursiva.

3.3.4 Multiplicação recursiva

A função `multiplicaRecursivo` implementa a etapa principal do algoritmo de divisão e conquista.

A função recebe duas matrizes de mesma ordem e retorna uma nova matriz correspondente ao produto entre elas.

O caso base ocorre quando a ordem das matrizes é igual a um. Nessa situação, a multiplicação é realizada diretamente, conforme apresentado no Código 2.

```
if (n == 1) {
    Matriz *C = createMatriz(1);

    if (C != NULL)
        C->dados[0][0] = A->dados[0][0] * B->dados[0][0];

    return C;
}
```

Código 2: Caso base da multiplicação recursiva.

Para matrizes de ordem superior a um, são criadas quatro submatrizes de cada operando. Em seguida, são realizadas oito chamadas recursivas para calcular os produtos necessários à construção dos quatro blocos da matriz resultante. Os produtos são somados dois a dois, conforme as expressões apresentadas na Seção 3, e os quatro blocos resultantes são combinados pela função `juntaMatriz`. A implementação também realiza a liberação das matrizes temporárias após sua utilização, evitando que as alocações intermediárias permaneçam ocupando memória desnecessariamente.

3.3.5 Função principal de multiplicação

A função `multiplicaMatriz` funciona como interface para a multiplicação recursiva.

Inicialmente, verifica as matrizes recebidas e determina a menor potência de dois maior ou igual à ordem original.

Quando necessário, são criadas matrizes auxiliares maiores, preenchidas com zeros, para permitir a divisão recursiva em quadrantes de mesma dimensão.

Após a execução de `multiplicaRecursivo`, a função constrói a matriz resultante com a dimensão original, copiando os elementos correspondentes e liberando as estruturas temporárias.

Dessa forma, o programa permite multiplicar matrizes quadradas de ordem arbitrária, mantendo a lógica recursiva concentrada em uma função específica.

3.3.6 Programa principal

O arquivo `main.c` coordena a execução do programa, realizando a leitura da ordem das matrizes, a criação das estruturas, a leitura dos elementos e a chamada da função `multiplicaMatriz`.

Após a multiplicação, o resultado é impresso e a memória alocada é liberada.

A implementação verifica possíveis falhas de leitura e alocação, encerrando a execução quando não é possível prosseguir corretamente.

3.4 Análise de complexidade

A análise de complexidade do algoritmo considera o número de operações realizadas durante a multiplicação recursiva de duas matrizes quadradas de ordem n, bem como a quantidade de memória auxiliar necessária para sua execução.

3.4.1 Complexidade de tempo

Em cada chamada da função `multiplicaRecursivo`, o algoritmo divide as duas matrizes em quatro submatrizes de ordem $n/2$, realiza oito multiplicações recursivas e combina os resultados obtidos.

A criação das submatrizes exige a cópia de elementos das matrizes originais, resultando em um custo de $\Theta(n^2)$.

As oito multiplicações recursivas são realizadas sobre matrizes de ordem $n/2$, produzindo o termo $8T(n/2)$.

Por fim, as operações de soma e combinação dos blocos resultantes também possuem custo $\Theta(n^2)$. Dessa forma, a relação de recorrência do algoritmo é dada por:

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 8T(n/2) + \Theta(n^2), & \text{se } n > 1. \end{cases} \tag{9}$$

Para resolver essa recorrência, aplica-se o Teorema Mestre, considerando sua forma geral:

$$T(n) = aT(n/b) + f(n). \tag{10}$$

Identificam-se os seguintes parâmetros:

$$a = 8, \qquad b = 2, \qquad f(n) = \Theta(n^2). \tag{11}$$

Calculando o termo de comparação do Teorema Mestre:

$$n^{\log_b a} = n^{\log_2 8} = n^3. \tag{12}$$

Como $f(n) = \Theta(n^2)$, verifica-se que:

$$f(n) = O(n^{3-\varepsilon}), \tag{13}$$

para $\varepsilon = 1$.
Portanto, a recorrência satisfaz o primeiro caso do Teorema Mestre, resultando em:

$$\boxed{T(n) = \Theta(n^3)}. \tag{14}$$

Assim, o algoritmo apresenta complexidade de tempo cúbica em relação à ordem das matrizes.

3.4.2 Complexidade de espaço

A implementação utiliza alocação dinâmica para armazenar as matrizes originais, as submatrizes e os resultados intermediários das operações recursivas.

Em cada chamada, são criadas matrizes auxiliares cuja quantidade total de elementos é proporcional a n^2.

Embora sejam realizadas oito multiplicações recursivas, essas chamadas são executadas sequencialmente. Dessa forma, a memória necessária para uma chamada recursiva pode ser reutilizada após seu encerramento.

O espaço máximo utilizado pela recursão pode ser representado pela relação:

$$S(n) = S(n/2) + \Theta(n^2). \tag{15}$$

Expandindo a recorrência, obtém-se:

$$S(n) = \Theta\left(n^2 + \frac{n^2}{4} + \frac{n^2}{16} + \cdots\right). \tag{16}$$

A expressão corresponde a uma progressão geométrica de razão $1/4$. Como a soma da série geométrica é limitada por uma constante, conclui-se que:

$$\boxed{S(n) = \Theta(n^2)}. \tag{17}$$

A profundidade da recursão é $\Theta(\log n)$, sendo o espaço ocupado pela pilha de chamadas assintoticamente inferior ao espaço utilizado pelas matrizes auxiliares.

3.4.3 Análise do preenchimento com zeros

Para matrizes cuja ordem não é uma potência de dois, a implementação utiliza matrizes auxiliares de ordem:

$$m = 2^{\lceil \log_2 n \rceil}. \tag{18}$$

Para todo $n \geq 1$, verifica-se que:

$$n \leq m < 2n. \tag{19}$$

Consequentemente, $m = \Theta(n)$.

A criação e a cópia dos elementos para as matrizes auxiliares possuem custo $\Theta(m^2)$, enquanto a multiplicação recursiva apresenta custo $\Theta(m^3)$.

Como $m = \Theta(n)$, o preenchimento com zeros não altera as ordens assintóticas de tempo e espaço.

Portanto, considerando o algoritmo completo, obtêm-se as seguintes complexidades:

$$\boxed{T(n) = \Theta(n^3)} \tag{20}$$

$$\boxed{S(n) = \Theta(n^2)}. \tag{21}$$

3.5 Experimentos e resultados

Para verificar o funcionamento do algoritmo de multiplicação de matrizes, foram realizados testes com diferentes dimensões e configurações de entrada.

Os testes tiveram como objetivo avaliar a correção dos resultados produzidos pela implementação, incluindo o caso base da recursão, matrizes de ordem que não é potência de dois e matrizes de dimensões maiores.

3.5.1 Metodologia dos testes

Os testes foram organizados em arquivos de entrada e saída, armazenados no diretório `tests` do projeto.

Para cada caso, foi definido um arquivo contendo a ordem das matrizes e seus elementos, juntamente com outro arquivo contendo o resultado esperado da multiplicação.

A execução dos testes foi automatizada por meio de um `Makefile`, utilizando o comando:

```
make test
```

O procedimento executa o programa para cada arquivo de entrada e compara a saída produzida com o resultado esperado, utilizando a ferramenta `diff`.

Um teste é considerado aprovado quando os dois arquivos apresentam o mesmo conteúdo.

3.5.2 Casos de teste

Foram utilizados quatro casos de teste, apresentados na Tabela 1.

Tabela 1: Casos de teste da multiplicação de matrizes.

Teste	Ordem	Descrição	Resultado
01	2	Multiplicação de duas matrizes com elementos constantes.	Aprovado
02	1	Verificação do caso base da recursão.	Aprovado
03	3	Multiplicação com preenchimento de zeros.	Aprovado
04	4	Multiplicação de uma matriz pela matriz identidade.	Aprovado

3.5.3 Teste 01 – Matrizes de ordem 2

O primeiro teste utiliza duas matrizes de ordem 2, definidas por:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}. \tag{22}$$

O resultado esperado é:

$$C = A \cdot B = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}. \tag{23}$$

Esse teste verifica a multiplicação recursiva de matrizes cuja ordem é uma potência de dois.

3.5.4 Teste 02 – Caso base

O segundo teste utiliza matrizes de ordem 1:

$$A = \begin{bmatrix} 5 \end{bmatrix}, \qquad B = \begin{bmatrix} 3 \end{bmatrix}. \tag{24}$$

O resultado esperado é:

$$C = \begin{bmatrix} 15 \end{bmatrix}. \tag{25}$$

Esse teste verifica o caso base da recursão, no qual a multiplicação é realizada diretamente entre os únicos elementos das matrizes.

3.5.5 Teste 03 – Ordem não potência de dois

O terceiro teste utiliza matrizes de ordem 3:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad B = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}. \tag{26}$$

O resultado esperado é:

$$C = \begin{bmatrix} 30 & 24 & 18 \\ 84 & 69 & 54 \\ 138 & 114 & 90 \end{bmatrix}. \tag{27}$$

Esse caso verifica o procedimento de preenchimento com zeros, pois as matrizes são ampliadas internamente para ordem 4 antes da multiplicação recursiva.

3.5.6 Teste 04 – Matriz identidade

O quarto teste utiliza uma matriz de ordem 4 multiplicada pela matriz identidade.

A matriz de entrada é:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}. \tag{28}$$

Como a segunda matriz é a identidade I_4, o resultado esperado é:

$$A \cdot I_4 = A. \tag{29}$$

Esse teste verifica se o algoritmo preserva corretamente os elementos da matriz quando multiplicada pela identidade.

3.5.7 Análise dos resultados

Os quatro testes funcionais foram executados com sucesso, produzindo resultados iguais aos valores esperados. Os testes contemplam diferentes situações de execução, incluindo o caso base da recursão, matrizes com ordem potência de dois e matrizes que exigem preenchimento com zeros. Os resultados obtidos fornecem evidências da correção da implementação para os casos avaliados. Entretanto, esses testes não constituem uma demonstração formal de correção para todas as entradas possíveis. A análise assintótica apresentada na Seção 3.4 estabelece o comportamento teórico do algoritmo, enquanto os testes realizados verificam seu funcionamento em entradas específicas.

3.6 Considerações da tarefa

A implementação da multiplicação de matrizes por divisão e conquista permitiu aplicar os conceitos de recursividade, decomposição de problemas e análise de complexidade estudados na disciplina de Projeto e Análise de Algoritmos. A utilização de um Tipo Abstrato de Dados (TAD) para representar as matrizes contribuiu para a organização do código, separando as operações de criação, leitura, impressão e liberação de memória da lógica de multiplicação.

A estratégia adotada consiste na divisão das matrizes em quatro submatrizes, seguida da realização de oito multiplicações recursivas e da combinação dos resultados parciais. Uma decisão importante da implementação foi a utilização de preenchimento com zeros para permitir a multiplicação de matrizes cuja ordem não é uma potência de dois. Dessa forma, o algoritmo pode receber matrizes quadradas de diferentes dimensões sem modificar sua estrutura recursiva. A análise teórica demonstrou que a solução apresenta complexidade de tempo $\Theta(n^3)$ e complexidade de espaço $\Theta(n^2)$. Os testes funcionais realizados apresentaram os resultados esperados para os casos avaliados, incluindo matrizes de ordem 1, matrizes de ordem potência de dois e matrizes que exigem preenchimento com zeros.

Como limitação, destaca-se a necessidade de alocar e copiar diversas submatrizes temporárias durante a execução, o que introduz custos adicionais de memória e processamento. Embora a estratégia de divisão e conquista organize a solução de forma recursiva, a implementação com oito multiplicações por etapa mantém a mesma ordem de complexidade de tempo do algoritmo convencional de multiplicação de matrizes. Portanto, a solução desenvolvida atende ao objetivo de aplicar a técnica de divisão e conquista à multiplicação de matrizes, permitindo relacionar a implementação prática com sua análise matemática de complexidade.

4 Tarefa 2 – Algoritmo de Dijkstra utilizando vetores

4.1 Descrição do problema

A segunda tarefa consiste na implementação do algoritmo de Dijkstra para determinar os caminhos mínimos a partir de um vértice de origem em um grafo direcionado e ponderado, utilizando vetores como estrutura de armazenamento.

Um grafo pode ser representado por $G = (V, E)$, em que V corresponde ao conjunto de vértices e E ao conjunto de arestas. Cada aresta $(u, v) \in E$ possui um peso não negativo $w(u, v)$, que representa o custo de deslocamento entre os vértices u e v.

Dado um vértice de origem $s \in V$, o objetivo é determinar, para cada vértice $v \in V$, a menor distância possível entre s e v.

O custo de um caminho $P = (v_0, v_1, \ldots, v_k)$ é definido pela soma dos pesos das arestas que o compõem:

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}). \tag{30}$$

A distância mínima entre a origem s e um vértice v é definida por:

$$\delta(s, v) = \min_{P \in \mathcal{P}(s,v)} w(P), \tag{31}$$

em que $\mathcal{P}(s, v)$ representa o conjunto de caminhos entre s e v. Quando não existe um caminho entre esses vértices, considera-se que $\delta(s, v) = \infty$.

A entrada do programa utiliza o formato DIMACS para problemas de caminhos mínimos. Nesse formato, são informados a quantidade de vértices, a quantidade de arestas e os respectivos vértices de origem, destino e peso de cada aresta.

A saída apresenta, para cada destino alcançável, a distância mínima calculada e a sequência de vértices que compõe o caminho correspondente. Para destinos inalcançáveis, a distância é representada como infinita.

A implementação deve permitir o processamento de grafos de diferentes dimensões, considerando a utilização de vetores para armazenar as informações do grafo e os dados necessários à execução do algoritmo.

4.2 Estratégia de solução

Para resolver o problema dos caminhos mínimos, foi utilizado o algoritmo de Dijkstra, que emprega uma estratégia gulosa para determinar as menores distâncias entre um vértice de origem e os demais vértices de um grafo com pesos não negativos.

A estratégia consiste em manter, para cada vértice, uma estimativa da menor distância conhecida a partir da origem. Inicialmente, a distância do vértice de origem é definida como zero, enquanto as distâncias dos demais vértices são consideradas infinitas.

Em cada iteração, o algoritmo seleciona o vértice ainda não visitado que apresenta a menor distância estimada. Como os pesos das arestas são não negativos, essa escolha permite estabelecer definitivamente a menor distância até o vértice selecionado.

Após a seleção, são examinadas as arestas que partem desse vértice, verificando se existe um caminho de menor custo até seus vértices adjacentes. Esse procedimento é denominado relaxamento de arestas.

Considerando uma aresta (u, v) de peso $w(u, v)$, a distância estimada de v é atualizada quando:

$$d[u] + w(u, v) < d[v]. \tag{32}$$

Nesse caso, realiza-se a atualização:

$$d[v] = d[u] + w(u, v). \tag{33}$$

Além da atualização da distância, o vértice u é registrado como predecessor de v, permitindo reconstruir posteriormente o caminho mínimo encontrado.

O processo é repetido até que todos os vértices alcançáveis tenham sido visitados. Caso não exista um vértice não visitado com distância finita, o algoritmo é encerrado, pois os vértices restantes não são alcançáveis a partir da origem.

4.2.1 Representação do grafo

Para armazenar o grafo, foi utilizada uma representação baseada em vetores indexados, na qual os vértices e as arestas são mantidos em estruturas alocadas dinamicamente. Cada vértice armazena o índice de sua primeira aresta de saída. As arestas, por sua vez, armazenam o vértice de destino, o peso e o índice da próxima aresta associada à mesma origem.

Essa organização permite percorrer as arestas de saída de um vértice sem a necessidade de armazenar uma matriz de adjacência. A representação foi escolhida por permitir o armazenamento de grafos esparsos com consumo de memória proporcional à quantidade de vértices e arestas.

4.2.2 Seleção dos vértices

A seleção do próximo vértice a ser processado é realizada por meio de uma busca sequencial em um vetor de distâncias.

Em cada iteração, todos os vértices são examinados para identificar aquele que ainda não foi visitado e apresenta a menor distância estimada.

Essa abordagem dispensa a utilização de estruturas auxiliares mais complexas, como filas de prioridade, mas exige uma varredura dos vértices a cada seleção.

Consequentemente, a etapa de seleção apresenta custo linear em relação ao número de vértices, influenciando diretamente a complexidade total do algoritmo.

4.2.3 Reconstrução dos caminhos

Para permitir a apresentação dos caminhos mínimos, foi utilizado um vetor de predecessores. Sempre que o relaxamento de uma aresta produz uma distância menor para determinado vértice, seu predecessor é atualizado. Após a execução de Dijkstra, o caminho entre a origem e um destino pode ser reconstruído percorrendo os predecessores a partir do destino até alcançar a origem.

Como essa reconstrução ocorre na ordem inversa, os vértices são armazenados temporariamente e posteriormente apresentados na sequência correta. Dessa forma, a solução permite obter tanto as distâncias mínimas quanto os caminhos correspondentes para os vértices alcançáveis.

4.3 Implementação

A implementação foi desenvolvida utilizando estruturas de dados alocadas dinamicamente para representar o grafo e armazenar as informações necessárias à execução do algoritmo de Dijkstra.

O código foi organizado em arquivos de cabeçalho, responsáveis pela definição das estruturas e declaração das funções, e arquivos de implementação, que contêm as operações sobre o grafo, o algoritmo de caminhos mínimos e o programa principal.

Os arquivos `vertice.h` e `aresta.h` definem as estruturas utilizadas para representar os vértices e as arestas. O arquivo `grafo.h` define a estrutura do grafo e declara suas operações, enquanto `grafo.c` implementa as funções de criação, inserção de arestas, leitura e liberação de memória.

Os arquivos `dijkstra.h` e `dijkstra.c` contêm, respectivamente, as declarações e a implementação das funções relacionadas ao cálculo dos caminhos mínimos. Por fim, o arquivo `main.c` coordena a leitura dos argumentos, o processamento do grafo e a apresentação dos resultados.

4.3.1 Estruturas de dados

Para representar o grafo, foram definidas estruturas específicas para vértices, arestas e para o próprio grafo.

A estrutura `Vertice` armazena o identificador de um vértice e o índice de sua primeira aresta de saída, conforme apresentado no Código 3.

```
1 typedef struct {
2     int id;
3     int primeiraAresta;
```

```
} Vertice;
```

Código 3: Estrutura utilizada para representar um vértice.

O campo `id` identifica o vértice, enquanto `primeiraAresta` indica a posição da primeira aresta de saída no vetor de arestas do grafo.

Quando um vértice não possui arestas de saída, esse campo recebe o valor -1.

A estrutura `Aresta`, apresentada no Código 4, armazena o destino, o peso e o índice da próxima aresta associada à mesma origem.

```
typedef struct {
    int destino;
    int peso;
    int proxima;
} Aresta;
```

Código 4: Estrutura utilizada para representar uma aresta.

O campo `destino` identifica o vértice de chegada, enquanto `peso` representa o custo da aresta.

O campo `proxima` permite percorrer as demais arestas de saída do mesmo vértice, utilizando índices do vetor. O valor -1 indica o término dessa sequência.

A estrutura `Grafo` reúne os vetores de vértices e arestas, além das informações necessárias para controlar suas dimensões.

```
typedef struct {
    int numVertices;
    int numArestas;
    int capacidadeArestas;
    Vertice *vertices;
    Aresta *arestas;
} Grafo;
```

Código 5: Estrutura utilizada para representar o grafo.

O campo `numVertices` representa a quantidade de vértices do grafo, enquanto `numArestas` armazena a quantidade de arestas inseridas.

O campo `capacidadeArestas` indica a capacidade máxima do vetor de arestas alocado inicialmente.

Os ponteiros `vertices` e `arestas` permitem acessar as estruturas armazenadas dinamicamente.

Como os identificadores dos vértices no formato DIMACS começam em 1, o vetor de vértices é alocado com $V + 1$ posições, permitindo o acesso direto pelo identificador.

Além da representação do grafo, foi definida a estrutura `ResultadoDijkstra`, utilizada para armazenar os resultados do algoritmo.

```
typedef struct {
    int numVertices;
    int origem;
    int64_t *distancia;
    int *visitado;
    int *predecessor;
} ResultadoDijkstra;
```

Código 6: Estrutura utilizada para armazenar os resultados de Dijkstra.

O campo `distancia` armazena as menores distâncias conhecidas a partir do vértice de origem, utilizando inteiros de 64 bits para representar os custos acumulados.

O vetor `visitado` identifica os vértices cujas menores distâncias já foram estabelecidas pelo algoritmo.

O vetor `predecessor` registra o vértice anterior no caminho mínimo encontrado, permitindo reconstruir posteriormente a sequência de vértices entre a origem e cada destino.

Os campos `numVertices` e `origem` armazenam, respectivamente, a quantidade de vértices e o identificador do vértice inicial utilizado na execução.

Essa organização separa a representação do grafo dos resultados do processamento, permitindo que as estruturas sejam manipuladas e liberadas de forma independente.

4.3.2 Operações básicas do grafo

Para realizar a construção e manipulação do grafo, foram implementadas funções responsáveis pela criação da estrutura, inserção de arestas, leitura dos dados de entrada e liberação da memória alocada.

A função `criarGrafo` recebe a quantidade de vértices e a capacidade máxima de arestas, realizando a alocação dinâmica das estruturas necessárias. O vetor de vértices é alocado com $V + 1$ posições, permitindo que os identificadores sejam utilizados diretamente como índices. Inicialmente, cada vértice recebe seu identificador e o campo `primeiraAresta` é definido como -1, indicando que não existem arestas de saída registradas. O vetor de arestas é alocado com capacidade definida a partir das informações presentes no cabeçalho do arquivo de entrada.

A função `inserirAresta` recebe os identificadores dos vértices de origem e destino, além do peso da aresta, e realiza sua inserção na estrutura do grafo.

Antes da inserção, são verificadas a validade dos vértices, a não negatividade do peso e a disponibilidade de espaço no vetor de arestas.

Cada nova aresta é armazenada na próxima posição disponível do vetor, utilizando o campo `numArestas` como índice.

A associação entre as arestas de saída de um mesmo vértice é realizada por meio dos campos `primeiraAresta` e `proxima`, conforme apresentado no Código 7.

```
int indice = grafo->numArestas;

grafo->arestas[indice].destino = destino;
grafo->arestas[indice].peso = peso;
grafo->arestas[indice].proxima =
    grafo->vertices[origem].primeiraAresta;

grafo->vertices[origem].primeiraAresta = indice;
grafo->numArestas++;
```

Código 7: Inserção de uma aresta utilizando índices.

Nesse procedimento, a nova aresta passa a ocupar a primeira posição da sequência de arestas de saída do vértice de origem.

O campo `proxima` da nova aresta armazena o índice da aresta que anteriormente ocupava essa posição, preservando a ligação com as demais arestas do mesmo vértice. Essa estratégia permite inserir uma aresta em tempo constante, sem deslocar os elementos já armazenados no vetor.

A função `lerGrafo` é responsável pela leitura dos dados fornecidos pela entrada padrão, utilizando o formato DIMACS. Nesse formato, as linhas iniciadas por `c` representam comentários, enquanto a linha iniciada por `p sp` informa a quantidade de vértices e arestas do grafo.

As linhas iniciadas por `a` descrevem as arestas, apresentando os identificadores dos vértices de origem e destino e o respectivo peso.

Um exemplo simplificado desse formato é apresentado no Código 8.

```
c Grafo de exemplo
p sp 5 6
a 1 2 10
a 1 3 3
a 3 2 4
a 2 4 2
a 3 4 8
a 4 5 1
```

Código 8: Exemplo de entrada no formato DIMACS.

A leitura é realizada linha a linha. Após identificar o cabeçalho do problema, a função cria o grafo com as dimensões informadas e utiliza `inserirAresta` para registrar as arestas encontradas.

Durante o processamento, são verificadas possíveis inconsistências nos dados, incluindo identificadores inválidos, pesos negativos e divergências entre a quantidade declarada e a quantidade de arestas efetivamente inseridas.

Por fim, a função `destruirGrafo` realiza a liberação dos vetores de vértices e arestas, seguida da liberação da própria estrutura `Grafo`.

Essa operação permite recuperar a memória alocada dinamicamente após o processamento, evitando que as estruturas permaneçam ocupando recursos desnecessariamente. A separação dessas operações em funções específicas contribui para a organização do código e permite que a representação do grafo seja utilizada independentemente da implementação do algoritmo de Dijkstra.

4.3.3 Implementação do algoritmo de Dijkstra

A função `dijkstra` implementa o procedimento principal para determinar os caminhos mínimos a partir de um vértice de origem em um grafo com pesos não negativos.

A função recebe um ponteiro para a estrutura `Grafo` e o identificador do vértice de origem, retornando uma estrutura `ResultadoDijkstra` contendo as distâncias calculadas, os predecessores e as informações dos vértices visitados.

Inicialização dos vetores Inicialmente, são alocados os vetores necessários ao processamento do algoritmo.

O vetor `distancia` armazena as menores distâncias conhecidas a partir da origem. Seus elementos são inicializados com `INT64_MAX`, utilizado para representar uma distância infinita.

O vetor `predecessor` é inicializado com -1, indicando que ainda não existe um predecessor definido para cada vértice.

O vetor `visitado` é inicializado com zero, indicando que nenhum vértice foi processado.

Após essas inicializações, a distância do vértice de origem é definida como zero, conforme apresentado no Código 9.

```
for (int i = 1; i <= grafo->numVertices; i++) {
    resultado->distancia[i] = INT64_MAX;
    resultado->predecessor[i] = -1;
    resultado->visitado[i] = 0;
}

resultado->distancia[origem] = 0;
```

Código 9: Inicialização dos vetores do algoritmo de Dijkstra.

Essa configuração estabelece as condições iniciais necessárias para o cálculo das menores distâncias, considerando que apenas a origem possui distância conhecida no início da execução.

Seleção do vértice de menor distância Em cada iteração, o algoritmo procura o vértice ainda não visitado que apresenta a menor distância estimada.

Essa seleção é realizada por meio de uma busca sequencial, percorrendo todos os vértices armazenados no grafo.

O procedimento é apresentado no Código 10.

```
int u = -1;
int64_t menorDistancia = INT64_MAX;

for (int v = 1; v <= n; v++) {
            if (!resultado->visitado[v] && resultado->distancia[v] <
                menorDistancia) {
                menorDistancia = resultado->distancia[v];
                u = v;
            }
        }

if (u == -1)
    break;

resultado->visitado[u] = 1;
```

Código 10: Seleção do vértice de menor distância.

A variável `u` armazena o identificador do vértice selecionado, enquanto `menor` mantém a menor distância encontrada durante a busca. Caso nenhum vértice não visitado possua distância finita, a variável `u` permanece igual a -1, encerrando o processamento.

Quando um vértice é selecionado, sua posição no vetor `visitado` é marcada com o valor 1, indicando que sua menor distância foi estabelecida definitivamente.

Relaxamento das arestas Após selecionar um vértice, o algoritmo percorre suas arestas de saída para verificar se é possível reduzir as distâncias estimadas dos vértices adjacentes.

A navegação é realizada utilizando o índice da primeira aresta do vértice e o campo `proxima` das estruturas `Aresta`.

O procedimento de relaxamento é apresentado no Código 11.

```
int indice = grafo->vertices[u].primeiraAresta;

 while (indice != -1) {
            Aresta aresta = grafo->arestas[indice];

            int v = aresta.destino;
            int peso = aresta.peso;

            if (!resultado->visitado[v] && resultado->distancia[u] <=
                INT64_MAX - peso) {
                int64_t novaDistancia = resultado->distancia[u] + peso;

                if (novaDistancia < resultado->distancia[v]) {
                    resultado->distancia[v] = novaDistancia;
                    resultado->predecessor[v] = u;
                }
            }

            indice = aresta.proxima;
        }
```

Código 11: Percurso e relaxamento das arestas.

Para cada aresta (u, v), verifica-se se o caminho que passa pelo vértice `u` apresenta custo inferior à distância anteriormente registrada para `v`.

Quando essa condição é satisfeita, a distância de `v` é atualizada e seu predecessor passa a ser o vértice `u`.

A condição que utiliza `INT64_MAX - peso` evita que a soma entre a distância acumulada e o peso da aresta ultrapasse o limite do tipo inteiro utilizado.

Além disso, o relaxamento considera apenas vértices ainda não visitados, pois as menores distâncias dos vértices já processados foram estabelecidas anteriormente.

Encerramento do algoritmo O processamento continua enquanto existirem vértices não visitados com distância finita, respeitando o limite de iterações correspondente à quantidade de vértices do grafo. Ao final, a estrutura `ResultadoDijkstra` contém as menores distâncias encontradas a partir da origem e os predecessores necessários para reconstruir os caminhos correspondentes. Os vértices inalcançáveis permanecem com distância igual a `INT64_MAX`, permitindo identificá-los durante a apresentação dos resultados. A função retorna um ponteiro para a estrutura de resultados, que pode ser utilizada pelas funções de impressão e posteriormente liberada da memória.

4.3.4 Reconstrução dos caminhos

Após a execução do algoritmo de Dijkstra, as menores distâncias a partir do vértice de origem são armazenadas no vetor `distancia`. Entretanto, para apresentar a sequência de vértices que compõe cada caminho mínimo, é necessário utilizar as informações registradas no vetor `predecessor`.

A função `imprimirCaminho` é responsável por reconstruir e apresentar o caminho entre o vértice de origem e um destino específico.

O procedimento inicia no vértice de destino e percorre sucessivamente seus predecessores até alcançar o vértice de origem.

Durante esse percurso, os identificadores dos vértices são armazenados em um vetor auxiliar, pois a sequência obtida encontra-se na ordem inversa do caminho original.

O procedimento de reconstrução é apresentado no Código 12.

```
int atual = destino;
int tamanho = 0;

while (atual != -1 && tamanho < resultado->numVertices) {
    caminho[tamanho++] = atual;
    atual = resultado->predecessor[atual];
}
```

Código 12: Reconstrução do caminho por predecessores.

Após o preenchimento do vetor `caminho`, seus elementos são percorridos em ordem inversa, permitindo apresentar a sequência correta de vértices, desde a origem até o destino. Por exemplo, considerando um caminho mínimo formado pelos vértices $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5$, a reconstrução por predecessores produz inicialmente a sequência $5, 4, 2, 3, 1$.

A inversão dessa sequência permite obter o caminho na ordem correta. Antes da reconstrução, a função verifica se o destino é válido e se existe um caminho a partir da origem. Quando a distância armazenada para o destino corresponde a `INT64_MAX`, considera-se que o vértice é inalcançável, não sendo possível apresentar um caminho até ele.

4.3.5 Apresentação de todos os caminhos

Além da reconstrução de um caminho específico, foi implementada a função `imprimirTodosCaminhos`, responsável por apresentar os resultados para todos os vértices de destino, exceto a própria origem.

A função percorre os identificadores dos vértices em ordem crescente, consultando as distâncias calculadas e reconstruindo os respectivos caminhos por meio do vetor de predecessores.

Para os vértices alcançáveis, a saída utiliza o formato:

```
[origem,destino](distancia) caminho
```

Código 13: Formato de saída dos caminhos mínimos.

Como exemplo, considerando o grafo de teste utilizado na implementação, o caminho mínimo entre os vértices 1 e 5 é apresentado da seguinte forma:

```
[1,5](10) 1 3 2 4 5
```

Código 14: Exemplo de saída de um caminho mínimo.

Nesse formato, os valores entre colchetes identificam os vértices de origem e destino, o valor entre parênteses representa a distância mínima calculada e a sequência seguinte apresenta os vértices que compõem o caminho. Para vértices inalcançáveis, a distância é representada por `INF`, indicando que não existe um caminho entre a origem e o destino.

A padronização da saída permite comparar os resultados produzidos pelo programa com os caminhos mínimos disponibilizados nos arquivos de referência utilizados durante a etapa de validação.

Dessa forma, as funções de reconstrução e impressão complementam o algoritmo de Dijkstra, permitindo apresentar não apenas as distâncias mínimas, mas também os caminhos correspondentes.

4.3.6 Programa principal

O arquivo `main.c` é responsável por coordenar a execução do programa, realizando a interpretação dos argumentos fornecidos pela linha de comando, a leitura do grafo, a execução do algoritmo de Dijkstra e a apresentação dos resultados.

O programa permite informar o vértice de origem e, opcionalmente, um vértice de destino específico. Quando nenhum argumento é fornecido, o vértice 1 é utilizado como origem padrão.

A interpretação dos argumentos é realizada por meio da função `strtol`, que permite converter os valores recebidos como texto para números inteiros, verificando possíveis erros de conversão.

Após a validação dos argumentos, a função `lerGrafo` é chamada para construir a representação do grafo a partir dos dados fornecidos pela entrada padrão.

Em seguida, são verificadas as condições necessárias para a execução, incluindo a validade dos identificadores dos vértices de origem e destino.

Com o grafo carregado, a função `dijkstra` é executada para calcular os caminhos mínimos a partir do vértice de origem selecionado.

Medição do tempo de execução Para avaliar o desempenho da implementação, foi utilizada a função `clock`, disponibilizada pela biblioteca `time.h` da linguagem C.

A medição é realizada imediatamente antes e depois da chamada da função `dijkstra`, permitindo registrar o tempo de CPU consumido especificamente pelo algoritmo.

O procedimento utilizado é apresentado no Código 15.

```
clock_t inicio = clock();

ResultadoDijkstra *resultado =
    dijkstra(grafo, origem);

clock_t fimTempo = clock();

double tempo =
    (fimTempo - inicio) / (double)CLOCKS_PER_SEC;
```

Código 15: Medição do tempo de execução do algoritmo.

A diferença entre os valores retornados por `clock` representa a quantidade de unidades de tempo de CPU consumidas durante o intervalo medido.

A divisão por `CLOCKS_PER_SEC` converte essa diferença para segundos, permitindo registrar os tempos utilizados na análise experimental.

Como a medição envolve apenas a chamada do algoritmo, não são contabilizadas diretamente as etapas de leitura do grafo e impressão dos caminhos.

O tempo calculado é apresentado na saída de erro padrão, utilizando `stderr`, para evitar interferências no formato da saída principal do programa.

Apresentação dos resultados Após a execução do algoritmo, o programa verifica se foi informado um vértice de destino específico.

Quando um destino é fornecido, são apresentadas a menor distância calculada e a sequência de vértices que compõe o caminho correspondente.

Caso nenhum destino seja informado, a função `imprimirTodosCaminhos` é utilizada para apresentar os caminhos mínimos entre a origem e todos os demais vértices do grafo.

Os resultados são enviados para a saída padrão, permitindo seu redirecionamento para arquivos de texto utilizados na validação.

Por exemplo, a execução do programa com o grafo de Roma pode ser realizada por meio do seguinte comando:

```
./tarefa02.exe < tests/rome99c.gr > resultado_roma.txt
```

Código 16: Execução do programa com o grafo de Roma.

Nesse procedimento, o arquivo `rome99c.gr` é utilizado como entrada e os caminhos mínimos calculados são armazenados em `resultado_roma.txt`.

O tempo de execução do algoritmo permanece disponível no terminal, pois é apresentado separadamente pela saída de erro padrão.

Liberação de memória Ao final do processamento, o programa realiza a liberação das estruturas alocadas dinamicamente.

A função `destruirResultadoDijkstra` libera os vetores de distâncias, predecessores e vértices visitados, além da estrutura que armazena os resultados.

Em seguida, a função `destruirGrafo` libera os vetores de vértices e arestas, bem como a própria estrutura do grafo.

A implementação também verifica possíveis falhas durante a leitura dos dados, a validação dos argumentos e a execução do algoritmo, encerrando o programa quando não é possível prosseguir corretamente. Dessa forma, o programa principal integra as operações de leitura, processamento, apresentação dos resultados e gerenciamento de memória, mantendo a implementação do algoritmo de Dijkstra separada das operações de entrada e saída.

4.4 Análise de complexidade

A análise de complexidade considera o número de vértices V e o número de arestas E do grafo, avaliando a quantidade de operações realizadas pelo algoritmo de Dijkstra e a memória necessária para sua execução.

A implementação utiliza vetores indexados para representar o grafo e uma busca sequencial para selecionar o próximo vértice a ser processado.

4.4.1 Complexidade de tempo

O algoritmo de Dijkstra é executado por meio de um laço principal que realiza, no máximo, V iterações, uma vez que cada vértice pode ser selecionado e marcado como visitado apenas uma vez.

Em cada iteração, o algoritmo percorre o vetor de vértices para identificar aquele que ainda não foi visitado e possui a menor distância estimada.

Como essa busca examina até V vértices, seu custo é $\Theta(V)$ por iteração.

Considerando as V iterações, o custo total da seleção dos vértices é:

$$T_{\text{selecao}}(V) = V \cdot \Theta(V) = \Theta(V^2). \tag{34}$$

Após selecionar um vértice, o algoritmo percorre suas arestas de saída e realiza o procedimento de relaxamento.

Cada aresta é examinada, no máximo, uma vez durante a execução, pois as arestas de saída de um vértice são percorridas apenas quando esse vértice é selecionado.

Como cada relaxamento envolve um número constante de operações, o custo total dessa etapa é:

$$T_{\text{relaxamento}}(V, E) = O(E). \tag{35}$$

A inicialização dos vetores de distâncias, predecessores e vértices visitados exige uma varredura dos V vértices, apresentando custo $\Theta(V)$. Portanto, o tempo total do algoritmo pode ser representado pela expressão:

$$T(V, E) = \Theta(V) + O(V^2) + O(E). \tag{36}$$

Considerando o termo dominante, obtém-se:

$$\boxed{T(V, E) = O(V^2 + E)}. \tag{37}$$

Como a seleção do vértice de menor distância é realizada por busca sequencial, a implementação apresenta custo quadrático em relação ao número de vértices no pior caso.

Para grafos direcionados simples, em que $E = O(V^2)$, a expressão pode ser simplificada para:

$$\boxed{T(V) = O(V^2)}. \tag{38}$$

A utilização de uma fila de prioridade poderia reduzir o custo da seleção dos vértices. Entretanto, a implementação desenvolvida utiliza exclusivamente a busca sequencial, conforme a estratégia adotada para esta tarefa.

4.4.2 Complexidade de espaço

A implementação utiliza alocação dinâmica para armazenar os vértices, as arestas e os vetores auxiliares necessários ao algoritmo.

A representação do grafo possui um vetor de $V+1$ posições para armazenar os vértices e um vetor com capacidade para E arestas.

Dessa forma, o espaço necessário para armazenar o grafo é:

$$S_{\text{grafo}}(V, E) = \Theta(V + E). \tag{39}$$

O algoritmo de Dijkstra utiliza três vetores auxiliares:

- `distancia`: armazena as distâncias estimadas dos V vértices.
- `visitado`: identifica os vértices que já foram processados.
- `predecessor`: armazena os predecessores utilizados na reconstrução dos caminhos.

Cada um desses vetores possui tamanho proporcional à quantidade de vértices.

Consequentemente, o espaço auxiliar utilizado pelo algoritmo é:

$$S_{\text{Dijkstra}}(V) = \Theta(V). \tag{40}$$

Considerando a representação do grafo e os vetores auxiliares, o espaço total utilizado pela implementação é:

$$S(V, E) = \Theta(V + E) + \Theta(V). \tag{41}$$

Portanto:

$$\boxed{S(V, E) = \Theta(V + E)}. \tag{42}$$

Essa representação apresenta uma vantagem em relação à matriz de adjacência, que exige $\Theta(V^2)$ posições independentemente da quantidade de arestas existentes.

Para grafos esparsos, nos quais a quantidade de arestas é significativamente menor que V^2, a utilização de vetores indexados permite reduzir o consumo de memória.

4.4.3 Complexidade da reconstrução dos caminhos

Após a execução do algoritmo, os caminhos mínimos podem ser reconstruídos utilizando o vetor de predecessores.

Para reconstruir um caminho entre a origem e um destino, é necessário percorrer a sequência de predecessores até alcançar o vértice inicial.

Considerando um caminho formado por k arestas, o procedimento realiza $k+1$ acessos ao vetor de predecessores.

Assim, o custo da reconstrução de um caminho é:

$$T_{\text{caminho}}(k) = \Theta(k + 1). \tag{43}$$

Como um caminho mínimo pode ser representado sem ciclos, seu número de arestas é limitado por $V - 1$.

Portanto, no pior caso, a reconstrução de um único caminho apresenta:

$$T_{\text{caminho}}(V) = O(V). \tag{44}$$

Quando são reconstruídos e apresentados os caminhos para todos os destinos, o custo total pode atingir $O(V^2)$, pois cada caminho pode conter até V vértices.

O vetor auxiliar utilizado para armazenar temporariamente um caminho possui tamanho proporcional ao número de vértices, resultando em espaço auxiliar $O(V)$.

Dessa forma, a reconstrução dos caminhos não altera a ordem assintótica quadrática do tempo total no pior caso, considerando a implementação com busca sequencial.

4.4.4 Complexidade das operações do grafo

Além do algoritmo de Dijkstra, foram analisadas as operações utilizadas na construção e manipulação do grafo.

A função `criarGrafo` realiza a alocação dos vetores e a inicialização das informações dos vértices, apresentando custo de tempo $O(V)$.

A função `inserirAresta` armazena cada nova aresta na próxima posição disponível do vetor e atualiza os índices necessários para manter o encadeamento das arestas de saída.

Como essas operações não dependem da quantidade de vértices ou arestas já inseridas, cada inserção apresenta custo $\Theta(1)$.

A função `lerGrafo` realiza a leitura das informações dos vértices e das arestas do arquivo de entrada.

Considerando a leitura de um arquivo DIMACS com V vértices e E arestas, e desconsiderando o custo das linhas de comentário, o processamento apresenta custo $O(V + E)$.

Por fim, a função `destruirGrafo` libera os vetores de vértices e arestas e a própria estrutura do grafo.

A análise dessas operações demonstra que a representação por vetores indexados permite construir e armazenar o grafo de maneira eficiente, sem a necessidade de utilizar uma matriz de adjacência.

Considerando a construção do grafo, a execução de Dijkstra e a apresentação dos caminhos mínimos, a complexidade de tempo no pior caso permanece limitada por:

$$\boxed{T(V, E) = O(V^2 + E)}. \tag{45}$$

O espaço total utilizado permanece proporcional à quantidade de vértices e arestas:

$$\boxed{S(V, E) = \Theta(V + E)}. \tag{46}$$

4.5 Experimentos e resultados

Para avaliar o funcionamento e o desempenho da implementação do algoritmo de Dijkstra, foram realizados experimentos utilizando grafos de diferentes dimensões.

Os testes tiveram como objetivos verificar a correção dos caminhos mínimos calculados, avaliar o tempo de execução do algoritmo e observar o comportamento da implementação em função do crescimento do número de vértices e arestas.

4.5.1 Metodologia experimental

Os experimentos foram realizados utilizando grafos direcionados e ponderados, representados no formato DIMACS.

Inicialmente, foi utilizado um grafo pequeno, construído para verificar o funcionamento das operações básicas, a atualização das distâncias e a reconstrução dos caminhos mínimos.

Posteriormente, foram utilizados dois grafos de redes rodoviárias de dimensões distintas: um correspondente à cidade de Roma e outro à rede rodoviária de Nova York.

As características dos grafos utilizados são apresentadas na Tabela 2.

O grafo de Roma foi utilizado para avaliar o algoritmo em uma rede de menor dimensão, enquanto o grafo de Nova York permitiu observar seu comportamento em uma rede significativamente maior.

Em ambos os casos, o vértice de origem utilizado nos experimentos foi o vértice 1.

Procedimento de execução A implementação foi desenvolvida na linguagem C e compilada utilizando o compilador GCC, com as opções de compilação `-Wall`, `-Wextra`, `-std=c11` e `-O2`.

Os experimentos foram realizados a partir da execução do programa em terminal, utilizando o redirecionamento da entrada padrão para carregar os arquivos DIMACS.

Para cada grafo, o algoritmo foi executado cinco vezes, mantendo as mesmas condições de entrada.

O tempo de execução foi medido por meio da função `clock`, considerando exclusivamente o intervalo correspondente à execução da função `dijkstra`.

Dessa forma, os tempos registrados não incluem diretamente a leitura do arquivo de entrada nem a impressão dos caminhos mínimos.

As medições foram expressas em segundos e utilizadas para calcular a média aritmética e o desvio-padrão amostral dos tempos observados.

A média aritmética é definida pela Equação 47.

$$\bar{t} = \frac{1}{n} \sum_{i=1}^{n} t_i, \tag{47}$$

em que t_i representa o tempo registrado na execução i e n corresponde à quantidade de execuções realizadas.

O desvio-padrão amostral é calculado pela Equação 48.

$$s = \sqrt{\frac{\sum_{i=1}^{n} (t_i - \bar{t})^2}{n-1}}. \tag{48}$$

Essas medidas permitem avaliar tanto o tempo médio de execução quanto a variação entre as repetições realizadas.

Validação dos resultados Além da avaliação de desempenho, foi considerado um procedimento de validação dos caminhos mínimos calculados.

Inicialmente, o grafo pequeno foi utilizado para verificar manualmente as distâncias e os caminhos encontrados. Para o grafo de Roma, foi utilizado como referência o arquivo `spathsRome99c.gr`, contendo informações de caminhos mínimos associados ao grafo utilizado nos testes. Foi preparado um script em Python para auxiliar a comparação entre os resultados gerados pela implementação e os dados de referência.

O procedimento permite verificar a correspondência entre as distâncias calculadas e as distâncias esperadas, além de avaliar a consistência dos caminhos apresentados. A comparação das distâncias é especialmente importante porque diferentes caminhos podem apresentar o mesmo custo mínimo. Dessa forma, a validação não deve depender exclusivamente da igualdade entre as sequências de vértices, mas também do custo total dos caminhos.

Os resultados quantitativos dos experimentos são apresentados nas subsubseções seguintes.

4.5.2 Resultados dos experimentos

Os experimentos foram realizados com os grafos de Roma e Nova York, utilizando cinco execuções independentes para cada conjunto de dados. Os tempos registrados correspondem exclusivamente à execução do algoritmo de Dijkstra, sem considerar as etapas de leitura do grafo e apresentação dos caminhos mínimos.

Resultados para o grafo de Roma O primeiro conjunto de experimentos utilizou o grafo de Roma, composto por 3.353 vértices e 8.859 arestas.

Os tempos obtidos nas cinco execuções são apresentados na Tabela 3.

A média dos tempos registrados foi de aproximadamente 0,008362 segundos, enquanto o desvio-padrão amostral foi de aproximadamente 0,000948 segundos. Os resultados indicam que a implementação conseguiu processar o grafo de Roma em poucos milissegundos. Observa-se que as quatro primeiras execuções apresentaram tempos próximos, enquanto a quinta execução registrou um valor inferior.

Resultados para o grafo de Nova York O segundo conjunto de experimentos utilizou o grafo de Nova York, composto por 264.346 vértices e 730.100 arestas.

Os tempos registrados são apresentados na Tabela 4.

Para o grafo de Nova York, o tempo médio de execução foi de aproximadamente 44,024087 segundos, com desvio-padrão amostral de aproximadamente 1,338396 segundos.

Os tempos registrados apresentaram maior variação absoluta em comparação aos resultados obtidos para o grafo de Roma.

A menor medição foi de 42,966839 segundos, enquanto a maior foi de 46,213813 segundos.

O aumento significativo no tempo de execução está associado à maior dimensão do grafo e ao custo da busca sequencial utilizada para selecionar os vértices durante o algoritmo.

Resumo dos resultados A Tabela 5 apresenta uma comparação dos principais resultados obtidos nos experimentos.

Os resultados demonstram uma diferença expressiva entre os tempos de processamento dos dois grafos avaliados. Enquanto o grafo de Roma foi processado em uma fração de segundo, o grafo de Nova York exigiu aproximadamente 44 segundos de tempo de CPU, em média. Essa diferença evidencia a importância de considerar o número de vértices e a estratégia de seleção utilizada pelo algoritmo ao trabalhar com grafos de grande dimensão.

Uma análise mais detalhada da relação entre os tempos observados e a complexidade teórica da implementação é apresentada a seguir.

4.5.3 Comparação entre os grafos de Roma e Nova York

A comparação entre os resultados obtidos para os grafos de Roma e Nova York permite analisar o comportamento da implementação do algoritmo de Dijkstra diante do aumento da quantidade de vértices e arestas.

O grafo de Roma possui 3.353 vértices e 8.859 arestas, enquanto o grafo de Nova York possui 264.346 vértices e 730.100 arestas.

A razão entre as quantidades de vértices dos dois grafos é dada por:

$$R_V = \frac{V_{\text{NY}}}{V_{\text{Roma}}} = \frac{264346}{3353} \approx 78{,}84. \tag{49}$$

De maneira semelhante, a razão entre as quantidades de arestas é:

$$R_E = \frac{E_{\text{NY}}}{E_{\text{Roma}}} = \frac{730100}{8859} \approx 82{,}41. \tag{50}$$

Portanto, o grafo de Nova York possui aproximadamente 78,84 vezes mais vértices e 82,41 vezes mais arestas que o grafo de Roma.

Entretanto, o crescimento do tempo de execução foi consideravelmente superior.

Utilizando os tempos médios obtidos nos experimentos, a razão entre os tempos é calculada por:

$$R_T = \frac{\bar{t}_{\text{NY}}}{\bar{t}_{\text{Roma}}} = \frac{44{,}024087}{0{,}0083616} \approx 5265. \tag{51}$$

Assim, o tempo médio de execução para o grafo de Nova York foi aproximadamente 5.265 vezes maior que o tempo médio registrado para Roma.

Essa diferença pode ser relacionada à complexidade de tempo da implementação:

$$T(V, E) = O(V^2 + E). \tag{52}$$

Como a seleção do próximo vértice é realizada por busca sequencial, essa operação pode exigir até V comparações em cada uma das V iterações.

Consequentemente, o custo quadrático associado à quantidade de vértices tende a apresentar grande influência no tempo de execução para grafos de maiores dimensões.

Considerando apenas o crescimento do termo quadrático, a razão teórica entre os custos é dada por:

$$R_{V^2} = \left(\frac{V_{\text{NY}}}{V_{\text{Roma}}} \right)^2. \tag{53}$$

Substituindo os valores:

$$R_{V^2} = \left(\frac{264346}{3353} \right)^2 \approx 6215. \tag{54}$$

A Tabela 6 apresenta um resumo das razões de crescimento.

Observa-se que o crescimento experimental do tempo de execução apresenta a mesma ordem de grandeza do crescimento do termo quadrático V^2.

Esse comportamento é compatível com a análise de complexidade realizada anteriormente, na qual a busca sequencial dos vértices apresenta custo $O(V^2)$.

Tabela 2: Características dos grafos utilizados nos experimentos.

Grafo	**Vértices**	**Arestas**
Roma	3.353	8.859
Nova York	264.346	730.100

Tabela 3: Tempos de execução do algoritmo de Dijkstra para o grafo de Roma.

Execução	**Tempo (s)**
1	0,008882
2	0,008872
3	0,008771
4	0,008606
5	0,006677
Média	0,008362
Desvio-padrão	0,000948

Tabela 4: Tempos de execução do algoritmo de Dijkstra para o grafo de Nova York.

Execução	**Tempo (s)**
1	44,318441
2	42,966839
3	46,213813
4	43,582669
5	43,038672
Média	44,024087
Desvio-padrão	1,338396

Tabela 5: Resumo dos tempos de execução dos grafos avaliados.

Métrica	**Roma**	**Nova York**
Vértices	3.353	264.346
Arestas	8.859	730.100
Execuções	5	5
Tempo médio (s)	0,008362	44,024087
Desvio-padrão (s)	0,000948	1,338396

Tabela 6: Comparação das razões de crescimento entre os grafos.

Grandeza	**Razão aproximada**
Número de vértices	78,84
Número de arestas	82,41
Tempo médio de execução	5.265
Termo quadrático V^2	6.215

Entretanto, os valores não precisam coincidir exatamente, pois a complexidade assintótica descreve o comportamento do algoritmo em função do tamanho da entrada, desconsiderando constantes multiplicativas e fatores específicos da execução. Além disso, o número de vértices efetivamente processados, a estrutura das conexões do grafo e as condições do ambiente computacional podem influenciar os tempos observados. Deve-se considerar também que a comparação utiliza apenas dois grafos de dimensões diferentes, não sendo suficiente para determinar experimentalmente uma função exata de crescimento.

Apesar dessa limitação, os resultados são consistentes com o comportamento esperado para a implementação de Dijkstra com seleção sequencial dos vértices. A utilização de uma estrutura de fila de prioridade, como um heap binário, poderia reduzir o custo da seleção dos vértices, permitindo uma implementação com complexidade de tempo $O((V + E) \log V)$.

Dessa forma, os experimentos evidenciam que a escolha da estrutura utilizada para selecionar os vértices exerce influência significativa sobre o desempenho do algoritmo, principalmente em grafos com grande quantidade de vértices.

4.5.4 Análise dos resultados

Os experimentos realizados permitiram avaliar o comportamento da implementação do algoritmo de Dijkstra em grafos com diferentes quantidades de vértices e arestas. Os resultados demonstraram uma diferença significativa no tempo de execução entre os grafos de Roma e Nova York. Para o grafo de Roma, o algoritmo apresentou tempo médio de aproximadamente 0,008362 segundos, enquanto para o grafo de Nova York o tempo médio foi de aproximadamente 44,024087 segundos.

Esse aumento expressivo está relacionado principalmente à estratégia utilizada para selecionar o próximo vértice a ser processado. Na implementação desenvolvida, essa operação é realizada por meio de uma busca sequencial no vetor de vértices, apresentando custo $O(V)$ por seleção. Como essa busca pode ser repetida até V vezes, seu custo acumulado pode atingir $O(V^2)$. Dessa forma, o aumento da quantidade de vértices produz um impacto significativo no tempo de execução, mesmo quando o número de arestas cresce de maneira aproximadamente proporcional.

Influência da representação do grafo A utilização de vetores indexados para representar as listas de adjacência permitiu armazenar os grafos com espaço proporcional a $V + E$. Essa característica é especialmente relevante para os grafos utilizados nos experimentos, pois ambos apresentam uma quantidade de arestas relativamente pequena em comparação ao número máximo de conexões possíveis entre seus vértices. Uma representação baseada em matriz de adjacência exigiria espaço proporcional a V^2, independentemente da quantidade de arestas existentes.

A representação por vetores indexados também permite percorrer diretamente as arestas de saída de cada vértice, sem examinar posições correspondentes a arestas inexistentes.

Entretanto, essa estrutura não elimina o custo quadrático da implementação, pois a seleção do vértice de menor distância continua sendo realizada por busca sequencial. Assim, a representação do grafo contribui principalmente para a eficiência do armazenamento e do percurso das arestas, enquanto o método de seleção dos vértices determina o principal custo assintótico da implementação.

Variação dos tempos de execução As cinco execuções realizadas para cada grafo apresentaram variações nos tempos registrados. No grafo de Roma, o desvio-padrão amostral foi de aproximadamente 0,000948 segundos. Para o grafo de Nova York, o desvio-padrão amostral foi de aproximadamente 1,338396 segundos.

Essas diferenças podem estar associadas a fatores do ambiente computacional, como o escalonamento de processos, a utilização dos recursos do sistema e o comportamento da memória durante a execução.

Como a medição foi realizada utilizando a função `clock`, os valores representam o tempo de CPU consumido pelo trecho analisado, não necessariamente o tempo total decorrido entre o início e o fim do programa. Além disso, a medição foi limitada à execução da função `dijkstra`, não incluindo diretamente o tempo necessário para carregar o arquivo DIMACS ou imprimir os resultados.

Essa separação permite avaliar o desempenho do algoritmo sem incluir o custo das operações de entrada e saída.

Limitações dos experimentos Embora os resultados obtidos sejam compatíveis com a análise teórica, os experimentos apresentam algumas limitações.

A avaliação de desempenho foi realizada utilizando apenas dois grafos de redes rodoviárias, com dimensões significativamente diferentes. Consequentemente, não é possível determinar uma função exata de crescimento do tempo de execução apenas com essas medições. Além disso, as características estruturais dos grafos, como a distribuição das arestas e a quantidade de vértices alcançáveis a partir da origem, podem influenciar o número efetivo de operações realizadas.

A utilização de um conjunto maior de grafos, com quantidades progressivamente crescentes de vértices e arestas, permitiria avaliar o comportamento experimental de maneira mais detalhada. Também seria possível comparar a implementação desenvolvida com outras versões do algoritmo, especialmente aquelas que utilizam filas de prioridade para selecionar os vértices.

Considerações sobre o desempenho De maneira geral, os experimentos demonstraram que a implementação apresenta tempos reduzidos para grafos de menor dimensão, mas seu desempenho é significativamente afetado pelo crescimento da quantidade de vértices.

A comparação entre Roma e Nova York apresentou um crescimento experimental do tempo de execução compatível com a influência do termo quadrático presente na análise de complexidade. A representação do grafo por vetores indexados mostrou-se adequada para armazenar redes rodoviárias esparsas, mantendo o consumo de memória proporcional à quantidade de vértices e arestas. Entretanto, a busca sequencial utilizada na seleção dos vértices constitui uma limitação importante para o processamento de grafos de grande dimensão. Portanto, os resultados reforçam a importância da escolha das estruturas de dados e das estratégias de implementação na eficiência dos algoritmos de caminhos mínimos.

4.6 Considerações da tarefa

Nesta tarefa, foi desenvolvido um programa em linguagem C para determinar os caminhos mínimos a partir de um vértice de origem, utilizando o algoritmo de Dijkstra em grafos direcionados com pesos não negativos.

A implementação utilizou uma representação do grafo baseada em vetores indexados, na qual os vértices e as arestas são armazenados em estruturas alocadas dinamicamente. Essa abordagem permitiu representar as listas de adjacência por meio de índices, evitando a utilização de ponteiros individuais para conectar as arestas de um mesmo vértice.

O algoritmo foi implementado utilizando uma estratégia gulosa, selecionando a cada iteração o vértice não visitado com a menor distância estimada e realizando o relaxamento de suas arestas de saída. Além do cálculo das distâncias mínimas, foi implementado o armazenamento dos predecessores, permitindo reconstruir e apresentar os caminhos encontrados entre a origem e os demais vértices. A análise de complexidade demonstrou que a implementação apresenta tempo $O(V^2 + E)$, devido principalmente à busca sequencial utilizada para selecionar os vértices.

O espaço total necessário é $\Theta(V + E)$, considerando a representação do grafo e os vetores auxiliares utilizados pelo algoritmo.

Nos experimentos realizados, foram utilizados grafos de redes rodoviárias correspondentes a Roma e Nova York, com diferentes quantidades de vértices e arestas.

O tempo médio de execução foi de aproximadamente 0,008362 segundos para o grafo de Roma e 44,024087 segundos para o grafo de Nova York. A diferença entre os tempos observados foi compatível com a influência do custo quadrático da seleção sequencial dos vértices, conforme identificado na análise teórica. Os resultados também evidenciaram a importância da representação do grafo por vetores indexados, especialmente em redes esparsas, nas quais o armazenamento de uma matriz de adjacência exigiria uma quantidade significativamente maior de memória.

Entretanto, a implementação apresenta limitações de desempenho para grafos de grande dimensão, em razão do custo associado à seleção dos vértices. Como possibilidade de melhoria, poderia ser utilizada uma fila de prioridade baseada em heap binário, reduzindo a complexidade de tempo para $O((V + E) \log V)$. Outra possibilidade seria ampliar os experimentos para grafos de diferentes dimensões e características, permitindo uma avaliação mais detalhada do comportamento do algoritmo.

De maneira geral, a tarefa permitiu aplicar conceitos relacionados a algoritmos gulosos, estruturas de dados para representação de grafos, análise de complexidade e avaliação experimental de desempenho.

A implementação também demonstrou como diferentes decisões sobre estruturas de dados e operações fundamentais podem influenciar a eficiência de um algoritmo, mesmo quando sua estratégia principal permanece inalterada.

5 Tarefa 3 – Algoritmo de Dijkstra utilizando Listas Encadeadas

5.1 Descrição do problema

A terceira tarefa consiste na implementação do algoritmo de Dijkstra para determinar os caminhos mínimos em um grafo direcionado e ponderado, utilizando listas encadeadas como estrutura de representação das adjacências. O problema é equivalente ao apresentado na Tarefa 2, diferenciando-se principalmente pela estrutura de dados utilizada para armazenar as arestas do grafo. Considerando um grafo $G = (V, E)$, em que V representa o conjunto de vértices e E o conjunto de arestas, cada aresta $(u, v) \in E$ possui um peso não negativo $w(u, v)$. Dado um vértice de origem $s \in V$, o objetivo é determinar a menor distância entre s e os demais vértices alcançáveis, além de reconstruir os caminhos mínimos correspondentes.

Nesta implementação, cada vértice possui uma lista encadeada contendo suas arestas de saída. Cada nó da lista armazena o identificador do vértice de destino, o peso da aresta e um ponteiro para a próxima aresta. Diferentemente da representação por vetores indexados utilizada na Tarefa 2, as arestas são alocadas individualmente na memória e conectadas por ponteiros. O algoritmo deve utilizar busca sequencial para selecionar o próximo vértice de menor distância, sem o emprego de filas de prioridade, mantendo a mesma estratégia de processamento da implementação anterior. A entrada do programa utiliza o formato DIMACS, contendo a quantidade de vértices, a quantidade de arestas e as informações de origem, destino e peso de cada aresta. A saída apresenta as distâncias mínimas calculadas e os respectivos caminhos, representados pela sequência de vértices entre a origem e cada destino. Para vértices inalcançáveis, a distância é representada como infinita.

Além da implementação, a tarefa propõe a comparação entre diferentes formas de representação de grafos, considerando o tempo de execução, o espaço necessário para armazenamento e o custo de acesso às adjacências. Essa análise busca identificar as vantagens e limitações das listas encadeadas em relação aos vetores indexados e às matrizes de adjacência, especialmente para grafos esparsos e densos.

5.2 Estratégia de solução

Para resolver o problema, foi utilizada a mesma estratégia gulosa do algoritmo de Dijkstra apresentada na Tarefa 2, modificando apenas a representação das adjacências do grafo. A principal alteração consiste na utilização de listas encadeadas, nas quais cada vértice mantém um ponteiro para sua primeira aresta de saída. As demais arestas são acessadas sequencialmente por meio de ponteiros. Essa representação permite armazenar somente as arestas existentes, sem a necessidade de reservar espaço para todas as possíveis conexões entre os vértices.

5.2.1 Representação do grafo

O grafo é representado por um vetor de vértices, no qual cada elemento contém um ponteiro para o início de sua lista encadeada de adjacência. Cada nó da lista armazena o destino, o peso da aresta e um ponteiro para o próximo nó. A inserção de uma aresta é realizada no início da lista correspondente ao vértice de origem, permitindo que essa operação seja executada em tempo constante.

Diferentemente da Tarefa 2, na qual as conexões eram estabelecidas por índices de um vetor de arestas, a nova representação utiliza ponteiros para conectar os nós alocados dinamicamente.

5.2.2 Execução do algoritmo

O processamento mantém os mesmos vetores auxiliares de distâncias, predecessores e vértices visitados utilizados na implementação anterior. A seleção do próximo vértice continua sendo realizada por busca sequencial, sem utilização de fila de prioridade.

A principal diferença ocorre durante o relaxamento das arestas. Após selecionar um vértice, o algoritmo percorre sua lista encadeada de adjacência, seguindo os ponteiros até alcançar o final da lista. Para cada aresta encontrada, verifica-se se o caminho passando pelo vértice atual apresenta custo inferior à distância anteriormente registrada para o destino. Dessa forma, a lógica de Dijkstra permanece inalterada, permitindo comparar as duas implementações com foco nas diferenças entre as estruturas de armazenamento.

5.3 Implementação

A implementação foi desenvolvida em linguagem C, mantendo a organização modular utilizada na Tarefa 2. Os arquivos de cabeçalho e de implementação foram preservados, sendo modificadas principalmente as estruturas responsáveis pela representação do grafo e pelo percurso das adjacências.

5.3.1 Estruturas de dados

A principal modificação foi realizada nas estruturas `Vertice` e `Aresta`, que passaram a utilizar ponteiros para representar as listas encadeadas.

A estrutura `Aresta` armazena o destino, o peso e um ponteiro para a próxima aresta da lista.

```
typedef struct Aresta {
    int destino;
    int peso;
    struct Aresta *proxima;
} Aresta;
```

Código 17: Estrutura de uma aresta na lista encadeada.

A estrutura `Vertice` mantém o identificador do vértice e um ponteiro para sua primeira aresta de saída.

```
typedef struct {
    int id;
    Aresta *primeiraAresta;
} Vertice;
```

Código 18: Estrutura de um vértice com lista de adjacência.

A estrutura `Grafo` armazena a quantidade de vértices, a quantidade de arestas e um vetor de vértices.

```
typedef struct {
    int numVertices;
    int numArestas;
    Vertice *vertices;
} Grafo;
```

Código 19: Estrutura do grafo com listas encadeadas.

Diferentemente da Tarefa 2, não existe um vetor único para armazenar todas as arestas. Cada aresta é alocada individualmente e associada à lista do seu vértice de origem.

5.3.2 Inserção de arestas

A função `inserirAresta` foi adaptada para realizar a alocação dinâmica de cada nova aresta.

Após a alocação, o ponteiro `proxima` recebe o endereço da primeira aresta atual, e o ponteiro do vértice é atualizado para a nova aresta.

```
Aresta *novaAresta = malloc(sizeof(Aresta));

if (novaAresta == NULL)
    return 0;

novaAresta->destino = destino;
novaAresta->peso = peso;
novaAresta->proxima =
    grafo->vertices[origem].primeiraAresta;

grafo->vertices[origem].primeiraAresta = novaAresta;
grafo->numArestas++;
```

Código 20: Inserção de uma aresta no início da lista.

Esse procedimento possui custo constante, pois não é necessário percorrer a lista para inserir uma nova aresta. A função `destruirGrafo` também foi modificada para percorrer as listas de adjacência e liberar individualmente cada nó alocado.

5.3.3 Adaptação do algoritmo de Dijkstra

A estrutura `ResultadoDijkstra` e os procedimentos de inicialização, seleção de vértices e reconstrução dos caminhos foram mantidos.

A principal alteração na função `dijkstra` ocorreu no percurso das arestas de saída de cada vértice. Na implementação anterior, as arestas eram acessadas por índices de um vetor. Nesta versão, o percurso é realizado diretamente por ponteiros.

```
Aresta *aresta =
    grafo->vertices[u].primeiraAresta;

while (aresta != NULL) {
    int v = aresta->destino;
    int peso = aresta->peso;

    if (!resultado->visitado[v] &&
        resultado->distancia[u] <= INT64_MAX - peso &&
        resultado->distancia[u] + peso <
            resultado->distancia[v]) {

        resultado->distancia[v] =
            resultado->distancia[u] + peso;

        resultado->predecessor[v] = u;
    }

    aresta = aresta->proxima;
}
```

Código 21: Percurso e relaxamento das listas encadeadas.

O processamento continua até que o ponteiro `aresta` seja igual a `NULL`, indicando o final da lista de adjacência. Essa modificação mantém a lógica de relaxamento da Tarefa 2, alterando somente a forma de acesso às arestas.

5.3.4 Programa principal

O arquivo `main.c` manteve a mesma organização da Tarefa 2, realizando a leitura do grafo, a execução do algoritmo, a apresentação dos caminhos mínimos e a liberação da memória. A entrada continua utilizando o formato DIMACS, permitindo executar as mesmas instâncias de teste e comparar os resultados das duas implementações.

5.4 Análise de complexidade

A análise de complexidade considera o custo das operações realizadas sobre as listas encadeadas e o processamento do algoritmo de Dijkstra. Embora a representação do grafo tenha sido modificada em relação à Tarefa 2, a estratégia de seleção dos vértices e de relaxamento das arestas permanece a mesma.

5.4.1 Complexidade de tempo

A criação do grafo exige a inicialização dos V vértices, apresentando custo $\Theta(V)$. Cada aresta é inserida no início de uma lista encadeada, com custo $\Theta(1)$ por inserção. Assim, a construção completa do grafo possui complexidade:

$$T_{\text{construcao}}(V, E) = \Theta(V + E). \tag{55}$$

Durante a execução de Dijkstra, a seleção do vértice não visitado de menor distância continua sendo realizada por busca sequencial, com custo $O(V)$ por iteração.

Como podem ocorrer até V iterações, o custo total dessa etapa é $O(V^2)$.

O relaxamento das arestas é realizado percorrendo as listas encadeadas de adjacência. Como cada vértice é processado no máximo uma vez, cada aresta é examinada no máximo uma vez, resultando em custo $O(E)$.

Portanto, a complexidade temporal do algoritmo permanece:

$$\boxed{T(V, E) = O(V^2 + E)}. \tag{56}$$

Essa complexidade é equivalente à obtida na Tarefa 2, pois a alteração da representação das arestas não modifica o custo assintótico da seleção dos vértices nem do relaxamento das adjacências.

5.4.2 Complexidade de espaço

A representação por listas encadeadas utiliza um vetor com V vértices e um nó alocado dinamicamente para cada uma das E arestas. Cada nó armazena o destino, o peso e um ponteiro para a próxima aresta, ocupando uma quantidade constante de memória.

Dessa forma, o espaço necessário para armazenar o grafo é:

$$\boxed{S_{\text{grafo}}(V, E) = \Theta(V + E)}. \tag{57}$$

O algoritmo de Dijkstra utiliza três vetores auxiliares para armazenar distâncias, predecessores e informações dos vértices visitados, resultando em espaço auxiliar $\Theta(V)$.

Considerando a representação do grafo e as estruturas auxiliares, a complexidade espacial total é:

$$\boxed{S_{\text{total}}(V, E) = \Theta(V + E)}. \tag{58}$$

5.4.3 Comparação entre representações

As representações por vetores indexados e listas encadeadas apresentam a mesma complexidade assintótica de espaço, $\Theta(V+E)$, sendo adequadas para o armazenamento de grafos esparsos. Entretanto, existem diferenças em seus custos constantes. Na representação por vetores, as arestas são armazenadas em posições contíguas de memória, enquanto nas listas encadeadas cada aresta é alocada individualmente.

A utilização de ponteiros e alocações individuais pode aumentar o consumo de memória e prejudicar a localidade de acesso aos dados. Por outro lado, as listas encadeadas permitem inserir novas arestas sem a necessidade de reservar posições em um vetor de arestas. Uma matriz de adjacência, por sua vez, exige espaço $\Theta(V^2)$, independentemente da quantidade de arestas existentes.

Embora permita consultar diretamente a existência de uma aresta em tempo constante, essa representação pode apresentar elevado consumo de memória para grafos esparsos.

Assim, a escolha da representação depende da densidade do grafo, das operações realizadas e dos custos práticos de armazenamento e acesso às adjacências.

5.5 Experimentos e resultados

Para avaliar o desempenho da implementação por listas encadeadas, foram utilizadas as mesmas instâncias de grafos da Tarefa 2, permitindo comparar as duas representações sob condições semelhantes. Os experimentos foram realizados com os grafos de Roma e Nova York, disponíveis no formato DIMACS.

5.5.1 Metodologia dos testes

Inicialmente, a implementação foi validada utilizando um grafo pequeno com cinco vértices e seis arestas, verificando as distâncias mínimas e os caminhos produzidos pelo algoritmo.

Em seguida, foram realizados experimentos com duas instâncias de maior dimensão, apresentadas na Tabela 7.

Para cada instância, o algoritmo foi executado cinco vezes, registrando o tempo de processamento em segundos.

A medição foi realizada utilizando a função `clock()`, considerando somente a execução do algoritmo de Dijkstra, sem incluir a leitura do arquivo ou a impressão dos caminhos. Os tempos foram utilizados para calcular a média aritmética e o desvio-padrão amostral de cada implementação.

Tabela 7: Instâncias utilizadas nos experimentos.

Instância	Vértices	Arestas
Roma	3.353	8.859
Nova York	264.346	730.100

5.5.2 Resultados experimentais

A Tabela 8 apresenta os tempos registrados nas cinco execuções do grafo de Roma.

Tabela 8: Tempos de execução para o grafo de Roma (s).

Execução	Tarefa 2	Tarefa 3
1	0,008882	0,010191
2	0,008872	0,008271
3	0,008771	0,006622
4	0,008606	0,009292
5	0,006677	0,008857
Média	0,008362	0,008647
Desvio-padrão	0,000948	0,001331

Para o grafo de Nova York, os tempos obtidos são apresentados na Tabela 9.

Tabela 9: Tempos de execução para o grafo de Nova York (s).

Execução	Tarefa 2	Tarefa 3
1	44,318441	43,072907
2	42,966839	41,407636
3	46,213813	41,679474
4	43,582669	42,715275
5	43,038672	41,228662
Média	44,024087	42,020791
Desvio-padrão	1,338396	0,822979

5.5.3 Análise comparativa dos resultados

Os resultados mostram que as duas implementações apresentaram tempos de execução próximos, embora tenham sido observadas diferenças entre as instâncias avaliadas.

No grafo de Roma, a implementação por vetores indexados apresentou tempo médio de aproximadamente 0,008362 segundos, enquanto a implementação por listas encadeadas obteve 0,008647 segundos.

Isso corresponde a um tempo médio aproximadamente 3,41% maior para as listas encadeadas.

Entretanto, essa diferença é pequena em relação à variabilidade observada nas execuções, não permitindo afirmar uma superioridade consistente de uma representação para essa instância. No grafo de Nova York, a implementação por listas encadeadas apresentou tempo médio de 42,020791 segundos, enquanto a implementação por vetores indexados obteve 44,024087 segundos. Nesse caso, as listas encadeadas apresentaram uma redução aproximada de 4,55% no tempo médio de execução.

Apesar dessa vantagem observada, os resultados não demonstram que listas encadeadas sejam sempre mais eficientes, pois o desempenho também depende das características do grafo e do ambiente de execução. Como ambas as implementações utilizam busca sequencial para selecionar o próximo vértice, o custo quadrático dessa operação permanece predominante para instâncias com muitos vértices.

Assim, os experimentos indicam que a mudança na representação das adjacências pode influenciar o desempenho prático, mesmo quando a complexidade assintótica do algoritmo permanece inalterada.

5.6 Considerações da tarefa

A implementação do algoritmo de Dijkstra utilizando listas encadeadas permitiu analisar a influência da representação do grafo sobre o desempenho e a organização das estruturas de dados. Em relação à Tarefa 2, a principal modificação foi a substituição dos índices utilizados para conectar as arestas por ponteiros, permitindo a construção de listas de adjacência com nós alocados dinamicamente.

A análise teórica demonstrou que essa alteração não modifica a complexidade temporal do algoritmo, que permanece $O(V^2 + E)$, nem a complexidade espacial total, que permanece $\Theta(V + E)$. Nos experimentos realizados, a implementação por vetores indexados apresentou um tempo médio ligeiramente menor para o grafo de Roma, enquanto a representação por listas encadeadas obteve melhor desempenho médio para o grafo de Nova York. Esses resultados indicam que a representação das adjacências pode influenciar o desempenho prático, mesmo quando as implementações apresentam a mesma complexidade assintótica.

As duas representações são adequadas para grafos esparsos, pois armazenam apenas as arestas existentes. Entretanto, os vetores indexados apresentam a vantagem de utilizar armazenamento contíguo, enquanto as listas encadeadas oferecem maior flexibilidade para inserções individuais de arestas. Em comparação, as matrizes de adjacência permitem consultar diretamente a existência de uma aresta, mas exigem espaço $\Theta(V^2)$, tornando-se menos adequadas para grafos grandes e esparsos.

Portanto, para as instâncias avaliadas, as representações por vetores indexados e listas encadeadas constituem alternativas viáveis. A escolha entre elas deve considerar não apenas a complexidade assintótica, mas também a organização da memória, as operações realizadas e as características do grafo.

6 Tarefa 4 – Algoritmo de Dijkstra utilizando Fila de Prioridade

6.1 Descrição do problema

A quarta tarefa consiste em implementar uma nova versão do algoritmo de Dijkstra, utilizando uma fila de prioridade para otimizar a seleção do vértice de menor distância. O problema de caminhos mínimos permanece o mesmo das Tarefas 2 e 3, assim como o formato de entrada DIMACS e a apresentação das distâncias e dos caminhos encontrados.

A principal diferença está na substituição da busca sequencial por uma fila de prioridade, implementada por meio de um heap binário mínimo (*min-heap*). Além disso, devem ser contabilizadas as operações realizadas durante a execução, incluindo inserções, remoções, atualizações de distância e vértices processados.

O objetivo é avaliar o impacto dessa estrutura no desempenho do algoritmo, comparando os resultados com as implementações anteriores, especialmente em grafos de grandes dimensões.

6.2 Estratégia de solução

A solução mantém a estratégia gulosa do algoritmo de Dijkstra e a representação do grafo por listas encadeadas utilizada na Tarefa 3.

A principal alteração ocorre na seleção do próximo vértice a ser processado. Em vez de percorrer sequencialmente o vetor de distâncias, utiliza-se um min-heap, no qual a prioridade de cada vértice corresponde à menor distância conhecida a partir da origem. Inicialmente, apenas o vértice de origem é inserido na fila, com prioridade zero. A cada iteração, o vértice de menor prioridade é removido e suas arestas de saída são examinadas.

Quando um caminho de menor custo é encontrado, a distância do vértice de destino é atualizada. Caso ele ainda não esteja na fila, é inserido; caso contrário, sua prioridade é reduzida. Dessa forma, a fila permite selecionar e atualizar os vértices de maneira mais eficiente, eliminando a necessidade de realizar uma busca sequencial a cada iteração.

6.3 Implementação

A implementação foi desenvolvida em C, aproveitando as estruturas de grafo e as funções de leitura, impressão e liberação de memória da Tarefa 3. Foram adicionados os arquivos `fila_prioridade.h` e `fila_prioridade.c`, responsáveis pela definição e implementação do TAD `FilaPrioridade`.

A estrutura utiliza um vetor de elementos do heap, um vetor auxiliar de posições e campos para controlar seu tamanho e capacidade. Cada elemento armazena o identificador de um vértice e sua prioridade.

```
typedef struct {
    int vertice;
    int64_t prioridade;
} ElementoHeap;

typedef struct {
    ElementoHeap *elementos;
    int *posicao;
    int tamanho;
    int capacidade;
} FilaPrioridade;
```

Código 22: Estruturas utilizadas na fila de prioridade.

O vetor `posicao` permite localizar diretamente um vértice no heap, possibilitando a atualização de sua prioridade sem percorrer sequencialmente a estrutura.

As principais operações implementadas são `inserir`, `extrairMinimo`, `diminuirPrioridade` e `filaVazia`. As operações de inserção, remoção e atualização utilizam procedimentos auxiliares para preservar a propriedade de heap mínimo.

Na função `dijkstra`, a busca sequencial foi substituída pela operação `extrairMinimo`. Durante o relaxamento das arestas, são utilizadas as operações `inserir` e `diminuirPrioridade`, conforme a presença do vértice na fila.

Também foram adicionados à estrutura `ResultadoDijkstra` quatro contadores para registrar o número de inserções, remoções, atualizações de distância e vértices processados. Essas informações são apresentadas pelo programa principal, juntamente com o tempo de execução, para permitir a análise experimental.

6.4 Análise de complexidade

A análise considera um grafo direcionado $G = (V, E)$, em que V representa o número de vértices e E o número de arestas. A principal diferença em relação às implementações anteriores está na utilização de uma fila de prioridade baseada em heap binário mínimo.

6.4.1 Complexidade das operações do heap

O heap binário é organizado como uma árvore binária completa, armazenada em um vetor. Sua altura é proporcional a $\log V$.

As operações de inserção e remoção do elemento de menor prioridade podem exigir a reorganização dos elementos ao longo de um caminho da árvore, apresentando custo $O(\log V)$. A operação `diminuirPrioridade` também apresenta custo $O(\log V)$, pois utiliza o vetor auxiliar `posicao` para localizar o vértice em tempo constante e, quando necessário, reposicioná-lo no heap. A verificação de fila vazia é realizada em tempo constante, $O(1)$.

6.4.2 Complexidade de tempo

Inicialmente, a criação dos vetores auxiliares e da fila de prioridade apresenta custo $O(V)$. Durante a execução de Dijkstra, cada vértice alcançável é inserido e removido da fila no máximo uma vez. Como cada operação possui custo $O(\log V)$, o custo total dessas operações é:

$$O(V \log V). \tag{59}$$

Além disso, cada aresta é examinada no máximo uma vez durante o processamento de seu vértice de origem. Quando ocorre um relaxamento bem-sucedido, pode ser necessário atualizar a prioridade de um vértice no heap, operação com custo $O(\log V)$. Como existem E arestas, o custo total associado ao exame das arestas e às atualizações de prioridade é:

$$O(E \log V). \tag{60}$$

Somando os custos, obtém-se:

$$T(V, E) = O(V + V \log V + E \log V). \tag{61}$$

Portanto, para $V \geq 2$, a complexidade de tempo da implementação é:

$$\boxed{T(V, E) = O((V + E) \log V)}. \tag{62}$$

Essa complexidade representa uma melhoria em relação às implementações anteriores, nas quais a seleção sequencial do próximo vértice resulta em custo $O(V^2 + E)$.

6.4.3 Complexidade de espaço

A representação do grafo por listas de adjacência encadeadas requer espaço $O(V + E)$ para armazenar os vértices e as arestas. Os vetores de distâncias, predecessores e vértices visitados utilizam espaço $O(V)$. A fila de prioridade armazena no máximo V elementos, enquanto o vetor auxiliar de posições também possui tamanho proporcional a V.

Assim, o espaço auxiliar utilizado pelo algoritmo de Dijkstra é:

$$\boxed{S_{\text{aux}}(V) = O(V)}. \tag{63}$$

Considerando também o armazenamento do grafo, a complexidade total de espaço é:

$$\boxed{S(V, E) = O(V + E)}. \tag{64}$$

6.4.4 Comparação com as implementações anteriores

A Tabela 10 apresenta as complexidades das três implementações do algoritmo de Dijkstra desenvolvidas no trabalho.

Tabela 10: Comparação das implementações de Dijkstra.

Tarefa	Seleção do mínimo	Tempo total
2 – Vetores	$O(V)$	$O(V^2 + E)$
3 – Listas	$O(V)$	$O(V^2 + E)$
4 – Min-heap	$O(\log V)$	$O((V + E) \log V)$

Nas Tarefas 2 e 3, a busca sequencial pelo vértice de menor distância é realizada repetidamente, produzindo um custo quadrático em relação ao número de vértices. Na Tarefa 4, essa busca é substituída pela remoção do mínimo do heap, reduzindo o custo de seleção para $O(\log V)$. Essa alteração é especialmente vantajosa em grafos esparsos, nos quais o número de arestas é proporcional ao número de vértices, isto é, $E = O(V)$.

Nesse caso, a complexidade da Tarefa 4 é limitada por:

$$T(V, E) = O(V \log V), \tag{65}$$

enquanto as implementações anteriores mantêm o custo quadrático $O(V^2)$.

6.5 Experimentos e resultados

Os experimentos da Tarefa 4 tiveram como objetivo verificar o funcionamento do algoritmo de Dijkstra com fila de prioridade e avaliar seu desempenho em comparação com as Tarefas 2 e 3.

Foram utilizados grafos de teste pequenos e duas instâncias de redes viárias no formato DIMACS: Roma, com 3.353 vértices e 8.859 arestas, e Nova York, com 264.346 vértices e 730.100 arestas.

6.5.1 Testes de funcionamento

Inicialmente, foram executados testes com um grafo pequeno, um grafo desconexo e um grafo contendo arestas de peso zero. Os resultados obtidos foram compatíveis com as distâncias esperadas. No grafo pequeno, a saída da Tarefa 4 foi comparada com a da Tarefa 3 utilizando o comando `diff`, sem apresentar diferenças. A comparação também foi realizada no grafo de Roma, produzindo saídas idênticas para as duas implementações.

No grafo de Nova York, as saídas apresentaram diferenças na comparação textual. Como podem existir diferentes caminhos mínimos com o mesmo custo, ainda é necessário verificar se as distâncias calculadas pelas duas implementações são iguais.

6.5.2 Metodologia dos experimentos

Para avaliar o desempenho, cada grafo foi processado cinco vezes, utilizando o vértice 1 como origem. O tempo foi medido com a função `clock()`, considerando apenas a execução do algoritmo de Dijkstra, sem incluir a leitura do grafo ou a impressão dos caminhos. Os programas foram compilados em C com a opção de otimização `-O2`. Os testes foram executados em ambiente Linux por meio do WSL.

Para cada conjunto de execuções, foram calculados a média aritmética e o desvio-padrão amostral dos tempos.

Tabela 11: Tempos de execução da Tarefa 4.

Execução	Roma (s)	Nova York (s)
1	0,000324	0,049182
2	0,000500	0,056541
3	0,000319	0,050694
4	0,000720	0,052872
5	0,000330	0,053205
Média	0,0004386	0,0524988
Desvio-padrão	0,0001748	0,0027945

6.5.3 Tempos de execução

A Tabela 11 apresenta os tempos obtidos nas cinco execuções de cada grafo.

O grafo de Roma apresentou tempo médio de aproximadamente 0,439 milissegundo, enquanto Nova York apresentou tempo médio de aproximadamente 52,499 milissegundos. Embora Nova York possua uma quantidade significativamente maior de vértices e arestas, o algoritmo conseguiu processar essa instância em um tempo inferior a um décimo de segundo.

6.5.4 Operações da fila de prioridade

Além do tempo de execução, foram contabilizadas as operações realizadas durante o processamento.

A Tabela 12 apresenta os resultados observados, que permaneceram constantes nas cinco execuções de cada grafo.

Tabela 12: Operações registradas na Tarefa 4.

Operação	Roma	Nova York
Inserções	3.353	264.346
Remoções	3.353	264.346
Atualizações de distância	3.747	299.020
Vértices processados	3.353	264.346

Observa-se que cada vértice foi inserido e removido da fila uma única vez. O número de atualizações de distância foi superior ao de inserções porque alguns vértices tiveram suas prioridades reduzidas após a descoberta de caminhos de menor custo.

Esses resultados são compatíveis com a utilização de um heap indexado, que permite atualizar prioridades sem inserir cópias adicionais de um mesmo vértice.

6.5.5 Comparação de desempenho

A Tabela 13 compara os tempos médios das três implementações desenvolvidas.

Tabela 13: Comparação dos tempos médios de Dijkstra.

Tarefa	Roma (s)	Nova York (s)
2 – Vetores	0,0083616	44,0240868
3 – Listas	0,0086466	42,0207908
4 – Min-heap	0,0004386	0,0524988

Para quantificar a melhoria, foi calculado o fator de aceleração (*speedup*), definido pela razão entre o tempo médio da implementação anterior e o tempo médio da Tarefa 4:

$$S = \frac{T_{\text{anterior}}}{T_{\text{heap}}}. \tag{66}$$

A Tabela 14 apresenta os fatores de aceleração obtidos.

Em Roma, a implementação com heap apresentou um ganho de aproximadamente 19 vezes em relação às versões anteriores.

Tabela 14: Fatores de aceleração da Tarefa 4.

Comparação	Roma	Nova York
Tarefa 2 / Tarefa 4	19,06	838,57
Tarefa 3 / Tarefa 4	19,71	800,41

Em Nova York, o ganho foi ainda mais expressivo, alcançando aproximadamente 800 vezes em relação à Tarefa 3. Essa diferença está relacionada à complexidade das operações de seleção do próximo vértice. Nas implementações anteriores, essa seleção exige uma busca sequencial com custo $O(V)$. Na Tarefa 4, a utilização do heap reduz o custo da operação para $O(\log V)$, contribuindo para a melhoria do desempenho em grafos com muitos vértices.

6.5.6 Análise dos resultados

Os resultados experimentais mostram que a utilização de uma fila de prioridade baseada em heap binário reduziu significativamente o tempo de execução do algoritmo de Dijkstra.

O ganho foi especialmente expressivo no grafo de Nova York, caracterizado por uma grande quantidade de vértices e um número relativamente pequeno de arestas por vértice. Esse comportamento é coerente com a análise teórica, segundo a qual a implementação com heap apresenta complexidade $O((V + E) \log V)$, enquanto as implementações anteriores apresentam $O(V^2 + E)$.

Nos testes de Roma, os tempos muito pequenos produziram variações relativas mais elevadas entre as execuções. Portanto, os resultados de Nova York oferecem uma comparação experimental mais representativa do ganho obtido. De maneira geral, os experimentos demonstram que a escolha da estrutura de dados utilizada para selecionar os vértices influencia diretamente a eficiência do algoritmo, mesmo quando sua estratégia fundamental permanece inalterada.

6.6 Considerações da tarefa

A quarta tarefa permitiu avaliar o impacto da utilização de uma fila de prioridade na eficiência do algoritmo de Dijkstra. Mantendo a representação do grafo por listas encadeadas da Tarefa 3, a principal alteração foi a substituição da busca sequencial pelo uso de um heap binário mínimo para selecionar o próximo vértice.

A implementação utilizou um heap indexado, permitindo inserir, remover e atualizar prioridades em tempo $O(\log V)$. Com essa estrutura, a complexidade de tempo do algoritmo passou de $O(V^2+E)$, nas implementações anteriores, para $O((V+E) \log V)$, mantendo complexidade de espaço total $O(V+E)$.

Os experimentos demonstraram uma melhoria expressiva no desempenho. No grafo de Roma, a implementação com heap foi aproximadamente 19,7 vezes mais rápida que a Tarefa 3. No grafo de Nova York, o fator de aceleração foi de aproximadamente 800,4 vezes, evidenciando a vantagem da estrutura em grafos grandes e esparsos.

Os testes funcionais apresentaram os resultados esperados para os casos pequenos, e a comparação com a Tarefa 3 produziu saídas idênticas no grafo de Roma. Para Nova York, ainda é necessário verificar se as diferenças nos caminhos impressos correspondem apenas a alternativas de mesmo custo mínimo.

Como limitação, a implementação com heap exige operações adicionais de manutenção da estrutura e um vetor auxiliar para controlar as posições dos vértices. Entretanto, os resultados demonstraram que esse custo é compensado pela redução do tempo necessário para selecionar os vértices durante a execução.

Conclui-se que a escolha de estruturas de dados adequadas pode melhorar significativamente o desempenho de um algoritmo, mesmo sem modificar sua estratégia fundamental de solução.

7 Tarefa 5 – Título a definir

8 Tarefa 6 – Título a definir

9 Tarefa 7 – Título a definir

10 Tarefa 8 – Título a definir

11 Tarefa 9 – Título a definir

12 Considerações Finais

Referências